University of Warsaw
Faculty of Mathematics, Informatics and Mechanics

Jakub Bednarz
Student no. 406103

Investigating and Combining Approaches for Data-Efficient Model-based RL

Master's thesis
in MACHINE LEARNING

Supervisor:
dr. Jacek Cyranka
Instytut Informatyki

Warsaw, Nov 2024

Abstract

[Abstract]

Keywords

...

Thesis domain (Socrates-Erasmus subject area codes)

11.4 Artificial Intelligence

Subject classification

...

Tytuł pracy w języku polskim

Analiza i połączenie metod w wydajnym obliczeniowo uczeniu ze wzmocnieniem bazowanym na modelu

Contents

1. Introduction . . . 5

2. Preliminaries . . . 7
2.1. Reinforcement Learning . . . 7
2.1.1. Deep RL . . . 8
2.2. Sample-efficient RL . . . 8
2.2.1. Benchmarks for data-efficient RL . . . 8
2.3. Model-based RL . . . 8
2.3.1. Learning the model . . . 8
2.3.2. Using the model . . . 9
2.4. Data augmentation in RL . . . 10
2.5. Self-supervised RL . . . 10
2.6. Training recipes for data-efficient RL . . . 11
2.7. Integrated architectures . . . 11

3. Method . . . 13
3.1. Design of the framework and the empirical studies . . . 13
3.1.1. World model . . . 13
3.1.2. RL algorithms . . . 14
3.1.3. Training recipes . . . 14
3.1.4. Data augmentation . . . 15
3.1.5. Self-supervised RL . . . 15
3.2. Implementation . . . 15
3.2.1. Configuration . . . 16
3.2.2. Environments . . . 17
3.2.3. World model and RL modules . . . 18
3.2.4. Data loaders and model-free mode . . . 21
3.2.5. Training recipes . . . 21

4. Experiments . . . 23
4.1. Experimental setup . . . 23
4.2. Training recipe optimization . . . 25
4.3. Simple speedup test . . . 25
4.3.1. Decoupling model and RL optimization . . . 26
4.4. Investigating the learning dynamics of the agents . . . 27
4.4.1. Analyzing the world model . . . 27
4.4.2. Plasticity analysis . . . 29
4.4.3. Analyzing the RL component . . . 31

4.5. Auto-tuning update frequencies . . . 31
4.5.1. Decoupled adaptive ratio system . . . 32
4.6. Data augmentation . . . 34
4.7. Choice of the RL algorithm choice . . . 37
4.7.1. PPO Experiments . . . 38
4.7.2. SAC Experiments . . . 38
4.8. Combining the improvements . . . 42
4.8.1. Hyperparameter selection . . . 42
4.8.2. Atari-100k benchmark . . . 43

5. Discussion . . . 49

Chapter 1

Introduction

Reinforcement Learning (RL), a research field dedicated to developing intelligent autonomous agents, capable of learning to do tasks in various environments [60], has achieved remarkable progress in recent times. Through the use of deep learning and artificial neural networks, algorithms playing complex games at a super-human level, such as Atari [38], Go [57], Dota 2 [43], or StarCraft II [68], as well as controlling robots [44, 4] and finetuning Large Language Models from human feedback [45], have been developed.

However, most of the current state-of-the-art techniques have a significant drawback: they require excessive amounts of data (which, in this setting, we regard as the total time spent interacting with the environment) in order to achieve such performance. For example, the standard benchmark on a collection of Atari video games [2] assumes a total interaction budget of 200 million game frames, equivalent to a single person playing for $\approx$ 40 days without rest. For playing Dota 2, thousands of years in total are required. This shortcoming greatly limits real-world deployment of such methods. Simulation of the target environment is often not available, and even in the presence of such models, the inaccuracies cause the policies developed in simulation to not transfer completely to the real world. There may also arise additional safety considerations [71], forcing us to constrain the number of interactions to a minimum.

In order to remedy this problem, many approaches designed with *sample efficiency*, being the maximization of agent performance in a given (small) number of interactions with the environment, in mind, have been proposed. Arguably the most appealing method is to learn a model of the environment, in order to predict the future or possible future events, and learn the optimal policy without any interactions with the real environment. Indeed, a number of data-efficient architectures have been developed using the world models in this capacity, via lookahead search [50, 73] or learning policies purely from synthetic data [16, 18, 29, 21].

There have, however, been efforts conducted to render learning the policy sample-efficient without the use of environment modelling. One line of research, for example, attempts to simply optimize the training recipe, e.g. the number of optimization steps per single environment interaction, without changing either the RL algorithm used, or the architecture [8, 49]. Other works introduce data augmentation [32, 72], self-supervised learning [55] or using larger neural network backbones [21, 54].

All these have resulted in significant improvements in terms of sample efficiency. Many of these innovations have been developed independent of each other, however, and in many cases are orthogonal, meaning it is possible to envision a single RL agent combining many such strategies and modifications, possibly capable of reaching even higher performance than standalone algorithms. A similar feat has been achieved by the Rainbow agent [25], which

augments a base RL architecture DQN [38] with an array of small modifications, which turn out to provide significant boost to the base agent in terms of final performance. Works such as BBF [54] have shown, that such an approach can also be used to increase sample efficiency of the agents, in the model-free scenario.

The goal of this thesis is to investigate whether this can be achieved for sample-efficient model-based RL. To be precise, our goal is to take a model-based architecture, originally not designed with data efficiency in mind - specifically DreamerV2 [20] - and use the ideas developed in other works, mostly in the context of model-free RL, adapting them if necessary to suit the different architecture, to see whether one can achieve significant performance benefits without any major architectural interventions.

The rest of the thesis is structured as follows. First, we perform the literature review, in order to assess what are the existing methods for improving data efficiency of RL agents. We identify the use of world models, finetuning the training recipes, applying data augmentations, and changing the RL algorithms used in model-based RL as the most promising developments to incorporate into our design. We propose how to combine them into a single RL agent, and design empirical studies aimed at investigating the effects of the ablations, and the changes in terms of agent performance.

After having considered a number of existing RL frameworks, and deeming them unsuitable for the purpose of conducting our study, we design and implement a novel PyTorch-based [46] RL framework capable of facilitating our experiments. We describe how different design choices help with making the framework as flexible, yet user-friendly, as possible, and highlight the advantages our solution has over other model-based RL frameworks. We make the framework and all the code necessary to replicate the experiments conducted in this thesis publicly available at `https://github.com/vitreusx/rsrch`.

Then, we will follow up with the execution of the experiments, and the analysis of the results. We discover that the design of the training loop, and the use of improved RL modules yield greatest improvements in terms of the final performance of the agent. On the other hand, otherwise promising avenues, such as the use of data augmentation, prove ultimately unsuccessful in improving the algorithm, and can even significantly degrade the policies learned.

Having in mind the number of hyperparameters which such a modular design introduces, we also employ and analyze methods for auto-tuning some of them. Specifically, following [10], we implement and improve upon a mechanism for automatically adjusting optimization frequency based on the model validation loss.

Throughout, we provide a detailed analysis of the learning dynamics and the behavior of the trained agents, in order to elucidate the causes of the observed performance improvements and hits, as well as better direct our efforts towards further modifications to discuss and test.

Finally, we use these findings to implement an RL agent using these modifications in tandem, and verify that the combination yields a further improvement in terms of validation scores achieved. We then test the best-performing configuration on Atari-100k benchmark, a standard benchmark for visual sample-efficient RL algorithms, and compare it with other state-of-the-art solutions. We find, that (...)

In summary, our principal contributions are:

1. An open source, highly customizable and modular model-based RL framework.
2. An in-depth analysis of the effects of various ablations of a model-based architecture.
3. A sample-efficient model-based RL agent architecture, (reaching the performance comparable with state-of-the-art works?)

Chapter 2

Preliminaries

2.1. Reinforcement Learning

Reinforcement Learning is a field of artificial intelligence and machine learning concerned with constructing intelligent agents capable of learning what actions to take in an environment so as to maximize a scalar reward signal [60].

Formally, the environment can be represented by a Markov decision chain (MDP) in the form of a tuple $\langle \mathcal{S}, \mathcal{A}, p \rangle$ consisting of the state space $\mathcal{S}$, action space $\mathcal{A}$ and transition probability $p(s_{t+1}, r_t \mid s_t, a_t)$, where $r_t \in \mathbb{R}$ denotes the reward granted at that step. Additionally, in the case of partially observable environments, we have an observation space $\mathcal{O}$ and an observation probability $p(o_t \mid s_t)$. The agent, starting from an initial observation o_1, acts according to a probability distribution $\pi(a_t \mid o_{\leq t})$ conditioned on previous observations until it reaches a *terminal* state s_T. The goal of reinforcement learning and the RL agent is to maximize the expected sum of rewards (so called returns) obtained along the trajectory $J = \mathbb{E}\left\{\sum_{t=1}^{T-1} r_t\right\}$.

There are two further quantities of interest. The state value function $V^\pi(s)$ equal to the expected returns of an agent acting with a policy $\pi(a_t \mid s_t)$, when starting from a state s. The state-action value function $Q^\pi(s, a)$ (called thereafter Q function) equal to the expected returns of an agent acting with a policy π, when starting from a state s and having just performed action a. These value functions are usually γ-discounted: rather than using the returns $\mathbb{E}\left\{\sum_{t=1}^{T-1} r_t\right\}$, we consider $\mathbb{E}\left\{\sum_{t=1}^{T-1} \gamma^{t-1} r_t\right\}$ for $0 < \gamma < 1$. This allows us to use fixed-point arguments in the proofs of various RL theorems. It is also the version most often used by deep RL algorithms.

One approach towards learning behavior in an environment is *Q-learning*. The idea is as follows: we want to learn an optimal policy π^* such, that V^{π^*} is maximized for all states s. It is clear, that such a policy should choose a, which maximizes $Q^{\pi^*}(s, a)$, i.e. π^* should be deterministic, and return $\pi^*(s) = \arg\max_a Q^{\pi^*}(s, a)$. It can be shown, that such a policy, and such an induced Q-function $Q^{\pi^*} =: Q^*$ satisfy $Q^*(s, a) = \mathbb{E}_{s'}\{r + \gamma Q^*(s', \pi^*(s'))\}$. This recurrence relation allows us, in turn, to learn the value of Q^* through fixed-point iteration.

A different way to learn optimal behavior is to learn the policy directly, rather than construct it from Q-value estimates, by taking a gradient of the returns and using standard first-order optimization methods. Specifically, by the policy gradient theorem [61], we have $\nabla_\theta J = \mathbb{E}\left[\sum_t \Psi_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right]$ for an on-policy sequence $(s_{1:T+1}, a_{1:T}, r_{1:T})$, where Ψ_t can be e.g. total trajectory returns J, state-action value $Q(s_t, a_t)$, or an *advantage value* $Q(s_t, a_t) - V(s_t)$.

2.1.1. Deep RL

The advances in deep learning have allowed components such as Q or value functions, and policies, to be realized as expressive function approximators, leading to breakthroughs in settings hitherto considered unsolvable. In a foundational work, [38] have designed a Q-learning-based agent DQN, reaching human-level performance on a suite of Atari video games. Actor-critic methods, such as Deep Deterministic Policy Gradients (DDPG) [34] and Soft Actor Critic (SAC) [17] have extended this approach to continuous action spaces. Methods based on policy gradients, such as Trust Region Policy Optimization (TRPO) [53] and Proximal Policy Optimization (PPO) [52], have also been developed.

2.2. Sample-efficient RL

Works such as DQN have reached human-level performance in arcade by playing an equivalent of 38.5 days (200×10^6 frames with frame rate of 60 FPS). Even with a simulator available, the excessive number of data required by standard RL algorithms can prove troublesome, as in the case of OpenAI Five [43], which had to be trained for 10 months on an entire distributed system, and required development of sophisticated "surgery" techniques due to having to restart training at multiple points in time. In domains such as robotics, this kind of scaling quickly becomes prohibitively expensive and time-consuming. Thus, developing sample-efficient methods is necessary to allow the deployment of deep RL agents in real environments.

2.2.1. Benchmarks for data-efficient RL

In order to gauge progress in sample-efficient deep RL, a number of standardized benchmarks have been proposed.

Atari-100k For discrete control, the *de facto* standard benchmark is Atari-100k [29], a subset of 26 games from the full Atari Learning Environment [2] benchmark, limited to 400×10^3 game frames, equivalent to 100×10^3 agent steps, assuming the standard practice of repeating every agent's action 4 times.

DMC For image-based continuous control, the most widely used benchmark has been DeepMind Control suite [63] limited to either 100×10^3 or 500×10^3 agent interactions, as proposed in [29]. This setup has been used by e.g. [32, 59].

2.3. Model-based RL

Whereas a model-free RL agent seeks only to learn the policy π, either directly or indirectly through a Q function, a model-based agent seeks also to simulate the environment itself and be able to predict the future, in order to aid learning the policy, or avoid having to do it altogether [60].

2.3.1. Learning the model

Most model-based RL algorithms aim to predict future stats or observations. SimPLE [29] performs the transition in input (image) space - the UNet-like model receives input consisting of last K frames, as well as action conditioning, and is trained to output next frame. The

main issue with this approach is that encoding and decoding images in such a fashion proves to be computationally expensive. Indeed, the training time is 5 A100-days (as reported by [21]). Thus, other model often opt to perform modelling in the latent space. World Models [16], for example, uses a two-stage training recipe: visual inputs are first compressed into a latent vector using a Variational Autoencoder [31], whereafter a combination of an RNN and a "distribution head" model the transition probabilities in the latent space. In the context of PlaNet agent [19], the authors raise the issues surrounding modelling stochastic environments autoregressively, and propose the use of both deterministic and stochastic recurrent networks in order to deal with stochasticity without inducing the decay of the information stored in the state vector. This approach has been further refined in the Dreamer series of models [18, 20, 21] by e.g. using discrete latent states, employing better techniques for learning prior and posterior state distributions, improving critic learning through the use of value distributions [1]. IRIS [37] uses a VQ-VAE [42] to encode input observations into a sequence of discrete tokens, and a GPT-like Transformer architecture [66] to predict future tokens, along with rewards and episode-end indicators.

Model states do not necessarily need to correspond to real states - in the case of *value-equivalent models*, the states and the transition function are optimized to approximate the values to be obtained after executing a sequence of actions [58]. The idea is to construct an abstract MDP, in which planning is equivalent to planning in the real environment. This way, model capacity is wholly used for the purposes of learning the policy, as opposed to modelling potentially irrelevant details. TreeQN [11], for example, learns a differentiable, recursive, tree-structured model acting as a replacement for the Q function in arbitrary model-free RL algorithm. In MuZero [50], the model is optimized to jointly predict rewards, the action-selection policy, and the value function, leading to an agent capable of learning to play Atari, Go and chess at a superhuman level. EfficientZero [73] adapts MuZero for sample-efficient image-based RL, through addition of self-supervised consistency loss [6], refinements in reward and value prediction, and a method to correct off-policy issues when using replay buffer samples to compute value targets.

2.3.2. Using the model

One class of methods uses the model for *planning*; such techniques are also known as *lookahead methods*. Here, planning refers to any decision process which actively uses the world model. For example, in its simplest incarnation, Model-Predictive Control (MPC), at every time step, optimizes a sequence of actions $a_t, \ldots, a_{t+H-1}$ such that the expected total rewards $\mathbb{E}\left\{\sum_{k=0}^{H-1} r_{t+k}\right\}$ is maximized, and selects the first action a_t - at the next step, the procedure is performed again. The action sequence can be optimized using any zero-order method, like Monte Carlo (i.e. sampling the actions randomly and selecting the one with the best score), or through Cross-Entropy Method (CEM). A variation of this idea is to employ a Monte Carlo Search Tree in order to focus on more promising directions of exploration. In all cases, an auxiliary value function predictor can also be used in order to approximate the returns beyond the rollout horizon.

At the other end of the spectrum lay approaches which use model rollouts for learning the policy. Such rollouts can be used either as a data augmentation technique, or completely replace training on real data, instead learning behaviors entirely in the "imagination" via regular model-free RL algorithms. The former approach, for example, has been utilized in MBPO [27], whereas the Dreamer series of algorithms [18, 20, 21] follows the latter strategy.

2.4. Data augmentation in RL

Drawing from the common practice of using data augmentation (e.g. image transformations like rotation, random shifts) in supervised learning to improve performance and reduce overfitting, there exist also methods, which apply these techniques in the context of RL. RAD [33] apply transforms, such as cropping, translation, conversion to grayscale or color jitter, to SAC and PPO. DrQ [32] develops an RL-specific augmentation schema: beyond simply applying random transforms to observations, the authors also use the fact, that Q and state-value functions ought to be invariant with respect to the transforms to smooth them out by averaging across multiple samples. In robotics, domain randomization has become a crucial component of sim2real pipelines [64, 44].

2.5. Self-supervised RL

Standard deep RL methods, such as DQN or SAC, only utilize the reward signal in order to learn useful state representations to be used to learn the policy. Some authors have proposed employing techniques from the area of self-supervised learning (SSL) in order to combat this inefficiency.

Broadly speaking, self-supervised learning consists of adding additional *pretext tasks*, which are solved not for their own sake, but rather to provide learning signal for the downstream representations. Many kinds of pretext tasks have been suggested:

- For visual input, one option is to exploit inherent structure of images, via rotation prediction [14], colorization (reconstructing the image from a grayscale version) [74] or jigsaw (splitting an image into patches, randomizing the order and reconstructing the original image) [41].
- Contrastive learning (CL) approaches, based on the instance discrimination task, construct such representations, that views (augmentations, crops etc.) of the same instance ("positive pairs") are pulled together, and views of different instances ("negative pairs") are pulled apart [7, 51]. Some CL methods do not use negative pairs altogether, relying on momentum, batch normalization and projection heads to avoid collapse of the representations [15].
- Generative approaches reconstruct input from noisy or corrupted versions. Autoencoders, variational autoencoders (VAEs) and denoising VAEs [67] pass the (possibly augmented) input through a latent bottleneck in order to recover the original input. The corruption can also take form of masking patches of the input - this approach has proven especially suited to pre-training representations for NLP [9] and Vision Transformers [23].

Similarly, self-supervised RL techniques attempt to incorporate pretext tasks into regular RL pipelines. Arguably the simplest way is to apply SSL to augment learning of observation embeddings. For example, CURL [59] takes a base RL model (in this instance SAC) and uses MoCo [24] to help train the observation encoder via an auxiliary CL loss term. Some architectures, however, design the pretext task to be more RL-specific. SPR [55], for instance, treats a future state and a state predicted by a transition model from the past state and the sequence of actions in between as a positive pair. Masked World Models [56] use the paradigm of masking image patches and reconstructing them, with an important addition of also predicting rewards, which proves crucial for achieving high final performance.

2.6. Training recipes for data-efficient RL

A recent line of research has focused on achieving greater sample efficiency without any architectural changes - rather, the idea is to optimize the training recipe, which we define as the way optimization and environment interaction steps are organized throughout the training process. In some cases, as shown in [65], it is sufficient to simply increase the frequency of optimization steps per environment step. At sufficiently high replay ratios, the final performance of the agents starts to decrease. In [39], the authors analyze this phenomenon, and suggest that the fundamental cause of this is *primacy bias*, whereby the neural networks overfit to early interactions, causing the loss of ability to learn and generalize from more recent data. The proposed remedy is to periodically fully or partially reinitialize the networks used by the agent - this simple procedure is shown to significantly improve performance in the low-sample regime.

2.7. Integrated architectures

The main idea of this paper is inspired and motivated by Rainbow architecture [25]. In it, the authors took a base off-policy model-free RL algorithm, DQN [38], applied an array of modifications developed by other authors (specifically: double Q-learning [22], dueling networks [69], prioritized sampling [48], noisy networks [12], distributional RL [1] and multi-step TD targets [62]), adapted to fit together in a unified architecture, and achieved substantial improvements in terms of final performance of the agent.

In the context of sample-efficient reinforcement learning, a recently introduced Bigger-Better-Faster (BBF) [54] agent employs a similar strategy. The authors take the SR-SPR architecture [8] and outfit it with harder periodic resets, larger encoder backbones used, a *receding update horizon* [30], meaning a variable length of sequences used for value estimation, a discount factor γ schedule [13], and the use of weight decay, by using AdamW optimizer [35]. Through careful study and incorporation of these ablations, human-level performance with human-level sample efficiency is achieved.

The objective of this thesis is to investigate a similar scheme in the context of model-based RL. However, unlike Rainbow and BBF, the changes we propose to test are rather higher-level compared with e.g. increasing model size, or adding distributional value estimation, mostly being drawn from other data-efficient architectures. Also, due to increased complexity of model-based RL frameworks, a different kind of approach is needed in order to incorporate these changes.

Chapter 3

Method

3.1. Design of the framework and the empirical studies

We are now in a position to consider what modifications to apply, design the structure of the RL agent capable of supporting them, and specify what studies concretely should we, at a minimum, conduct. With this, we will be able to assess whether existing RL frameworks can be used to perform the empirical tests.

Of the existing benchmarks for sample efficiency, we target Atari-100k, a set of 26 video games for Atari 2600, with the environment interaction cap of 100×10^3, as the *de facto* standard in visual control.

3.1.1. World model

First, we need to choose what kind of a model-based RL architecture to implement, which would best suit our objectives. Although models trained to value equivalence have proven very sample-efficient, they are also tightly coupled, making introduction of arbitrary modifications either difficult, or infeasible. We will therefore focus on world models trained independently of any policy, with observation reconstruction and/or self-supervised auxiliary losses being the main drivers of optimization. The framework should allow swapping out of the world model implementations at will, and support adjusting all the hyperparameters thereof.

We decide to focus on a single world model architecture. It is, of course, reasonable to expect model improvements to lead to better behavioral policies. We would, however, rather shed light on the less-highlighted aspects of the design of an model-based RL algorithm beside the model alone. We select DreamerV2 [20] for a number of reasons:

1. As opposed to the next iteration of the algorithm, DreamerV3 [21], it has not been designed with sample efficiency in mind. Because of this, we believe it can serve as a proxy for arbitrary model architecture, with the recommendations in this paper being possibly applicable to future work in model-based RL.

2. Having high base performance, measured in the extended setting with an interaction budget of 50×10^6, we hope that we can reach performance comparable with state-of-the-art methods even in the setting with 100×10^3 interactions. For example, (Compute the requisite speedup factor to each e.g. BBF).

3. Compared with e.g. IRIS [37] or SimPLE [29], the world model used is computationally friendly, taking hours rather than entire days for a single training run to complete, allowing us to conduct a vast array of tests.

3.1.2. RL algorithms

Aside from the world model itself, the other major component of the architecture is the RL algorithm (or RL module) itself. The interaction between the two can be realized in a number of distinct ways:

1. Dreamer series takes the approach of training a (possibly arbitrary) model-free RL algorithm purely with simulated experience. The model and the RL modules are essentially independent of each other.

2. One may mix real and simulated data for the purposes of training the RL agent. For example, in MBPO, the authors use an on-policy method (specificially PPO), but augment the data with imaginary rollouts. This requires the model states to be of the same data type as the observations, which limits its usefulness in the context of visual control, as is the case when playing Atari games.

3. The agent module may not even require any training per se. For example, PETS and PlaNet use cross-entropy method (CEM) for selecting actions to perform by planning with the learned model.

Thus, ideally there needs to be an option for the RL module to use the world model actively, in order to support planning-based methods. It also needs to be possible to take any model-free RL algorithm, and be able to use it in the framework, possibly with some small modifications to account for the model-based setting.

In our tests, we will start with the RL algorithm supplied in the DreamerV2, namely a variant of Advantage Actor-Critic (A2C). Thus, our starting point will be an agent analogous to DreamerV2, with the same world model and RL algorithm. We shall examine the effect of replacing aforementioned module with two model-free algorithms: Soft Actor-Critic (SAC) and Proximal Policy Optimization (PPO). The choice is motivated by following factors:

1. Since the original RL algorithm in DreamerV2 uses, for Atari environments, vanilla policy gradients with Generalized Advantage Estimation, the most straightforward way to improve upon it is to replace it with an improved policy-gradient-based algorithm. We select PPO due to high performance and the ability to reuse simulated data, possibly increasing the overall computational performance.

2. Beyond that, we would like to investigate the behavior of Q-learning algorithms in a model-based setting. We select SAC as a strong baseline.

3.1.3. Training recipes

Beyond the network architecture, a major component of the design of sample-efficient agents is the structure of the training loop, meaning the order and frequency in which environment interaction, model optimization and RL module optimization occur. Indeed, works such as Data-Efficient Rainbow [65] or SR-SPR [8] prove, that ordinarily inefficient methods can be made vastly more data-efficient by refining the training process. Thus, it is necessary for the framework to accomodate specifying different kinds of training recipes. Ideally, the entire training recipe ought to be adjustable via a configuration file. Following the work on plasticity loss and applying periodic resets, it should also be possible to specify a relevant reset policy in the same fashion.

We will investigate the straightforward approach of tuning the model/RL update ratio. We would also like to assess, whether *decoupling* the two update frequencies has a positive effect on the performance. Given the number of hyperparameters to consider, it is also crucial to verify, whether a method for automatically tuning these parameters can be devised.

As for applying the periodic resets, we have found the required number of tests to be computationally infeasible, as the literature suggests different reset frequencies, and different reset strategies - one may reset all the parameters, or only the last K layers, and the resetting may be full or partial. This would, presumably, have to be conducted separately for the model and the RL modules, further increasing the extent of the hyperparamter sweep. Furthermore, prior work [28] suggests, that there may not necessarily be any clear training-time indicator of the phenomenon occuring. We will therefore limit our investigation to merely detecting, whether primacy bias occurs in our setting.

3.1.4. Data augmentation

Since applying data augmentation has proven to significantly improve data efficiency, the framework also needs to support their use. While some works recommend applying data augmentations in ways specifically suited for RL, such as Q function averaging, it requires modifications to be made in the RL module code, whereas one of our desiderata is the freedom in testing various RL algorithms. Thus, we only require the framework to provide an option to augment observations.

In our tests, we will follow the augmentation strategies devised for the DrQ agent [32].

3.1.5. Self-supervised RL

As regards the use of self-supervised learning and specifically pretext tasks, it is difficult to envision a completely generic and architecture-agnostic way to implement it. Moreover, it can be argued, that the use of an entire world model should render the benefits of adding auxiliary objectives and pretext tasks null. We will therefore not pursue this avenue of research. We may observe, however, that the considered framework should nevertheless allow for experiments involving these ablations, through introduction of new world models and/or RL modules incorporating such auxiliary objectives.

3.2. Implementation

We have considered a number of existing RL frameworks, with the goal of finding one which would support all the features we have described and facilitate the experiments. We have, however, found publicly available solutions to be inadequate for our study:

1. `google/dopamine` [5] - Only supports model-free algorithms. The algorithms are also written in JAX [3], meaning there is a greater barrier for entry for the users. Ultimately, we found it to be infeasible to convert it to a full-featured model-based framework.

2. `vwxyzjn/cleanrl` [26] - Only supports model-free algorithms. Also, due to a single-file design, it does not provide sufficient infrastructure for our experiments.

3. `facebookresearch/mbrl-lib` [47] - Only supports continuous state-action spaces. Moreover, we found the configurability and testing infrastructure (e.g. support of Tensorboard or Weights and Biases) lacking or nonexistent.

4. `opendilab/DI-engine` [40] - Although there is support for some model-based RL algorithms, such as MBPO or DreamerV3, they are not configurable to the extent we desire. Rather than being a single unified framework, it is a collection of disparate RL algorithms.

Thus, in order to facilitate the experiments, we have implemented a modular model-based RL framework in Python, using the PyTorch deep learning framework. The source code is accessible at `https://github.com/vitreusx/rsrch`, and is structured as a regular Python package. All the experiments can be performed with a (suitably configured) call to `python -m dreamerx`.

The main component responsible for running the experiments is the `Runner` class, responsible for instantiating the requisite neural networks, RL environments, replay buffers, data loaders, optimizers etc. We will now elucidate various aspects of the implementation, by following an execution flow of the program.

3.2.1. Configuration

First, we must load the configuration. Due to the variety of the experiments and ablations, that the system needs to accomodate, the framework needs to be highly configurable. Our implementation allows for the control of, among other things, *types* of the networks to be used - one may, for example, use IMPALA observation encoder rather than the original one - as opposed to only e.g. the layer sizes; and the entire structure of the training loop. We have found following properties to be crucial throughtout the development process:

1. There needs to be a single YAML file with default parameter values.

2. We need to be able to compose *presets*. These YAML entries are, in effect, overrides of the default config. These presets also need to be extensible and composable. For example, we may create a preset corresponding to a single experiment, containing the parameter values to be overriden, and want to create a variant of that experiment by changing a single value. We may also want to combine multiple presets, for example in order to copy environment specification from one preset, and training details from another.

3. The solution must allow for variable interpolation, meaning the capacity for one parameter to reference other values in the YAML file. The interpolation cannot be limited to strings alone. Also, we would like to be able to perform mathematical operations to compute certain parameter values on-the-fly. The interpolation ought to be performed after all the presets have been merged into the base configuration object.

4. The raw configuration object (a dictionary) needs to be convertible to a Python class, preferably a `dataclass`, in order to ensure type safety and increase the ease of development via the presence of the type hints.

Of the existing solutions, we have found `omegaconf` to be the one best matching our desiderata.

An abbreviated example of the preset system in action can be found in Listing 1. We have the main configuration file `dreamerx/config.yml` with default values, and an entry `data` with the description of replay buffer and data loader settings. Then, in another file `thesis/exp/presets.yml`, we can find two presets `data_aug.v1` and `data_aug.v2`, corresponding to two experiments. The `v1` experiment *extends* yet another experiment,

Listing 1 An example use of the preset system

```
--- dreamerx/config.yml
(...)
data:
  capacity: 2e6
  val_frac: 0.0
  loaders:
    dreamer_wm:
      batch_size: 16
      slice_len: 50
      subseq_len: [50, 50]
      prioritize_ends: true
      ongoing: false
      augment:
        type: none
(...)

-- thesis/exp/presets.yml
(...)
thesis.data_aug:
  v1:
    $extends: [adaptive_ratio.wm_v1]
    _rl_ratio: 4.0
    data.loaders.dreamer_wm:
      augment:
        type: drq
        drq:
          max_shift: 4

  v2:
    $extends: [baseline]
    data.loaders.dreamer_wm:
      augment:
        type: drq
        drq:
          max_shift: 4
(...)
```

`adaptive_ratio.wm_v1`, not shown due to space constraints, applying all the other overrides present there, setting an internal parameter `_rl_ratio`, and overriding the augmentation settings of a data loader. The end user, rather than having to supply the entire complex configuration via command line, can write it in a human-readable YAML format, and then simply invoke the `dreamerx` module with a preset option `-p thesis.data_aug.v1` to launch an experiment.

3.2.2. Environments

After parsing the configuration file, we must set up the RL environments, such as the Atari games, or DMC tasks. Our main design goal concerning the construction and use of the RL environments has been to separate the implementation details of the environments, and the rest of the project.

The main issue is as follows: the components further down the line, such as the actor networks or the world model, accept the PyTorch tensors as the input, whereas most RL libraries return Numpy arrays. The replay buffer(s) should also store Numpy arrays. We may note, that in some common cases, none of these "data formats" are not the same, even the replay buffer data and the data obtained from the environment. For example, many Atari agents utilize *frame stacking*, whereby last K emulator frames are passed to the agent. Clearly, we would like to store only the most recent one in the buffer, to avoid wasting memory - in fact, for a replay buffer of size 2×10^6 and grayscale observations of size 86×86, we would need $\approx$ 60G of memory to store all of them in an uncompressed form if not doing any memory optimization. Beyond this, we need to make sure, that the data transforms of the environment samples and buffer samples yield the same results. Usually, all these nuances are handled manually, but this approach introduces a lot of possibilities for inadvertantly adding hard-to-fix bugs.

Ideally, then, we would like to hide all of this from the user of the framework. The way we achieve this is through an *environment SDK*. The SDK's role is to expose a minimal interface for creating environments, performing interactions with them, and creating replay buffers to store the experience. The Python-esque pseudocode is provided in Listing 2. The idea is to provide an unified interface for acting in the environment(s) via a `VecAgent` protocol, and to hide the details concerning various conversions, data storage details etc. through `wrap_buffer` and `rollout` methods of the SDK.

Another crucial feature is the presence of `obs_space` and `act_space` member fields. They provide information concerning the format of the data received by the agent or sampled from the buffer. We provide a set of classes mirroring the `gym.spaces` classes, such as `Image` or `Box`, but for the PyTorch tensors, instead of Numpy arrays.

We provide SDKs for Atari Learning Environment (ALE), DeepMind Control Suite (DMC) and a generic one for `gymnasium` environments. For ALE and DMC, we use optimized, C++-based implementations provided by `envpool` library [70]. We may note that, despite `envpool` not sharing API with Gym environments, we do not need to make any changes to the rest of the code, like the training loop, showcasing one major advantage of using such SDKs over handling all the environment-related code manually.

3.2.3. World model and RL modules

In order to implement swapping out the world model and the RL algorithm implementations, we separate them out into replaceable modules with unified interfaces.

As regards the world model interface, it is difficult to devise a single common scheme for all possible methods we may envision. Some algorithms, for example, perform transitions in observation space, whereas others operate in a latent space. The model can either give the probability distribution over future states, or act in an implicit manner, e.g. by generating future states from noise via Diffusion Models.

We take the path of requiring the model to implement (1) an inference process $s_{t+1} \sim p(s_{t+1} \mid s_t, a_t, o_{t+1})$, (2) performing an imaginary rollout $\widetilde{s}_{t+1:t+H}, \widetilde{r}_{t+1:t+H} \sim p(\widetilde{s}_{t+1:t+H}, \widetilde{r}_{t+1:t+H} \mid s_t, a_{t:t+H-1})$, where H is the horizon length. This choice excludes the use of RL algorithms which explicitly require access to the transition probability distribution, but in practice none of the algorithms considered so far had such a requirement.

In the case of RL algorithm modules, we require the sampling of the next action, given the current state $a_t \sim \pi(a_t \mid s_t)$ - the actor is, in effect, memoryless, and we will rely on the world model to encode all the information necessary to perform optimal actions.

Listing 2 Pseudocode for environment SDKs

```
class VecAgent:
    def reset(self, indices, observations):
        """Receive first observations of the new episodes in the
        environments specified by the indices."""

    def policy(self, indices):
        """Choose actions to perform in given environments, as specified by
        the indices."""

    def step(self, indices, actions, observations):
        """Observe new observations and actions leading to them in the
        environments specified by the indices."""

class SDK:
    obs_space: Any
    """Observation space, in tensor format."""

    act_space: Any
    """Action space, in tensor format."""

    def make_envs(self, num_envs: int, **kwargs):
        """Create a number of environments. The implementation can be
        optimized, e.g. by vectorizing the environments."""

    def wrap_buffer(self, buffer):
        """Adapt a regular replay buffer into an SDK-aware version.
        When adding data obtained from the environment, it is converted to
        the buffer format. When sampling the data, the samples are converted
        to the tensor format."""

    def rollout(self, envs, agent: VecAgent):
        """Create a stream of interactions of an agent with the environment(s).
        The agent receives data in the tensor format, and should output actions
        in the tensor format as well."""
```

During the training process, additional code and networks are necessary to perform optimization. We split these responsibilities off to a separate `Trainer` class. For example, a `Trainer` for a world model may contain, aside from the optimizers, observation decoder, which is strictly speaking not necessary during inference. Similarly, a trainer for an actor-critic RL algorithm would contain the value and Q functions, as well as the target versions (if using double Q learning).

We provide implementations of DreamerV2's world model and RL algorithm - a variant of Advantage Actor-Critic. We follow the original source code for the agent `danijar/dreamerv2`. Let us note, that the authors' reference implementation as been written for the Tensorflow deep learning framework [36]. We have considered a number of open-source reimplementations of Dreamer in Pytorch, and found them to be inadequate in some ways:

1. `jsikyoon/dreamer-torch` is a very close translation of the original codebase, suffering from the same issues, mainly the code quality.

2. `jurgisp/pydreamer` contains a number of features not present in the original implementation, such as variable model update ratios, using an auxiliary actor-critic in the model, and importance-weighted advantage estimation.

3. `pytorch/rl` and `juliusfrost/dreamer-pytorch` only implement DreamerV1.

4. `Eclectic-Sheep/sheeprl` is a distributed RL framework.

As none of the considered options proved either usable as a Python package, or clear enough to "copy-paste" into the framework in the form of a standalone module, as described above, we have decided it would be better to reimplement it ourselves. Beyond merely replicating the Tensorflow code, we have added the ability to specify and change the encoder and decoder architectures for observations, rewards and termination signals. During development process, we have also found, that the eager-evaluation nature of Pytorch hurts the performance, compared to Tensorflow implementation, which uses JIT. Thus, we have also implemented a `torch.jit.script` version of the Recurrent State Space Module in PyTorch, which has increased the number of iterations per second by 25%.

Also, we provide implementations of two model-free algorithms: Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC). We base our implementations on the ones found in `vwxyzjn/cleanrl`, with a number of changes made to make them usable in the model-based context:

1. The implementation of SAC used a one-step TD target $y = r + \gamma V(s')$. In order to bring it in line with PPO and A2C, we change it to Generalized Advantage Estimation (GAE).

2. Since the "observations" given to the model are in fact world model states, we change the default convolutional encoders to the feedforward networks used in the A2C implementation.

3. Imaginary sequences generated by the model are unlike the regular sequences drawn from e.g. the replay buffer. Because we do not know *a priori* whether a sampled state is terminal or not - we may only use the terminality predictor of the model, which would return the probability that a state is terminal or not - we need to account for a degree of "fuzziness" (in the sense of fuzzy logic) of the latent states. The alternative could be to stop a particular rollout during the generation process, when the termination

probability exceeds a given threshold. In that instance, however, we would have to deal with batches of sequences of non-uniform length, which would likewise require changes to the implementation of the RL algorithm, and would likely be computationally inefficient, since we could not perform batched operations easily. We opt, therefore, for the former approach; this is also the approach taken in the DreamerV2's implementation of A2C. In practice, this nuance requires two implementation changes to be made: (1) per-item losses need to be multiplied by a weight factor, to prevent unreal states from affecting the optimization process, (2) GAE and other value estimation methods need to be changed in order to accept non-constant γ discount factors.

3.2.4. Data loaders and model-free mode

Given the replay buffer, the next step is to sample sequences from it, and collate them into a batch to be used for model or RL optimization. We extract this process into a separate *data loader* class, similar to PyTorch `torch.utils.data.DataLoader`. In this way, we make the training loop more readable, and more similar to the regular training loops used for the supervised tasks such as image classification or next-token prediction. The other advantage is the ability to experiment with different kinds of data loaders. We may, for example, use a variant with prioritized sampling, or with data augmentation added.

We can also use this functionality to implement a model-free debug mode. The idea is as follows: when we add a new RL algorithm to test, it would be wise to first test it in a model-free setting, to assess whether any possible agent performance issues are caused by the RL algorithm. This can be achieved in a following fashion:

1. Ordinarily, the RL data loader loads a batch of real-world sequences, performs inference process to obtain the state for each batch item and time step, and performs a rollout "in imagination" using the behavior policy and the model. We can, however, replace it by a regular data loader, loading batches of real-environment sequences.

2. There remains adjusting the various encoders and other networks to accept environment samples (e.g. images), as opposed to model states. This can be done simply through making changes to the configuration of the RL module.

3.2.5. Training recipes

After the stage of setting up neural networks, replay buffers, environments etc. we proceed to the training process proper. The main issue is the vastitude of the possible recipes - we could load and/or save checkpoints, load external data to replay buffer, want to use different schedules of performing model and RL training steps, perform validation epochs throughout the process and at the very end, reset network parameters at different time points etc. In order to resolve this issue in as straightforward and user-friendly a manner as possible, we have elected to allow the user to specify the recipe manually in the configuration file.

The relevant configuration field is `stages` - an example can be found in Listing 3. The task names (`prefill`, `do_wm_opt_step` etc.) correspond to functions of the `Runner` class. We specify the frequency or duration of various actions through `until` and `every` fields - for example, we can see that buffer prefill phase lasts until $P = 20 \times 10^3$ environment steps have passed. The training loop lasts until $T = 400 \times 10^3$ environment steps, and consists of validation epoch every 20×10^3 steps, model optimization step every 8 environment steps, RL optimization step every 4 environment steps etc. Finally, we do a final validation epoch and save the checkpoint.

Listing 3 An example `stages` configuration field.

```yaml
stages:
  - prefill:
      until: 20e3
  - train_loop:
      until: 400e3
      tasks:
        - do_val_epoch: ~
          every: 20e3
        - do_wm_val_step: ~
          every: { n: 16, of: wm_opt_step }
        - do_rl_val_step: ~
          every: { n: 16, of: rl_opt_step }
        - do_wm_opt_step: ~
          every: 8
        - do_rl_opt_step: ~
          every: 4
        - do_env_step
  - do_val_epoch
  - save_ckpt:
      full: false
      tag: final
```

Such an example experiment configuration would have been difficult to specify precisely, if we were to use pre-defined experiment presets, and would be far less comprehensible. In contrast, our solution allows the user (almost) full control over the structure of the training loop, and makes user's intentions far easier to discern.

Chapter 4

Experiments

4.1. Experimental setup

We are interested in evaluating performance on the Atari-100k benchmark. However, performing training runs on the entire set of 26 games and with a full range of RNG seeds for every modification is computationally infeasible. Thus, we will use a subset of these games and a reduced number of seeds as a proxy. To be specific, we shall limit ourselves to a choice of 3 environments and 5 RNG seeds. In order to properly gauge the effect of changes on the full set, we want to construct a subset of Atari-100k with following properties:

1. Since we are interested in *speeding up* the algorithm, we ought to select such environments, in which the method makes consistent progress in a given extended interaction budget. For example, given a maximum speedup target of 15×, we would focus on the horizon of $15 \cdot (400 \times 10^3) = 6 \times 10^6$ environment steps. If the method doesn't improve much beyond the performance of a random agent, or solves the task immediately, gauging the speedup factor becomes impossible.

2. The variance of the results with respect to different RNG seeds used should be minimized, so as to be able to derive the expected score with a limited number of RNG seeds with any degree of accuracy.

3. The subset should be as representative of the performance on the whole subset as possible.

First, let us filter out the environments which do not satisfy first two criteria. In the absence of standardized metrics which would inform our choice quantitatively, we resort to visual inspection of the performance curves for each environment in the first 6×10^6 steps. The relevant data can be found in Figure 4.1. Based on it, we select following games for further evaluation: `Amidar`, `Assault`, `Asterix`, `CrazyClimber`, `JamesBond`, `MsPacman` and `Pong`.

Next, we shall select out of this subset 3 games, which are most predictive of the performance on the complete 26-game benchmark. We follow the approach of (Atari-5), and use the source code provided by the authors to arrive at the final result. The three environments selected by the algorithm are: `Assault`, `CrazyClimber` and `MsPacman`.

We use the same environment settings (frame skip values, time limit etc.) as the original DreamerV2.

The original implementation of DreamerV2 is written in Tensorflow - we have ported the relevant source code (mainly the implementation of the world model and the actor critic) to Pytorch, resulting in slight changes in overall performance. It remains, however, comparable:

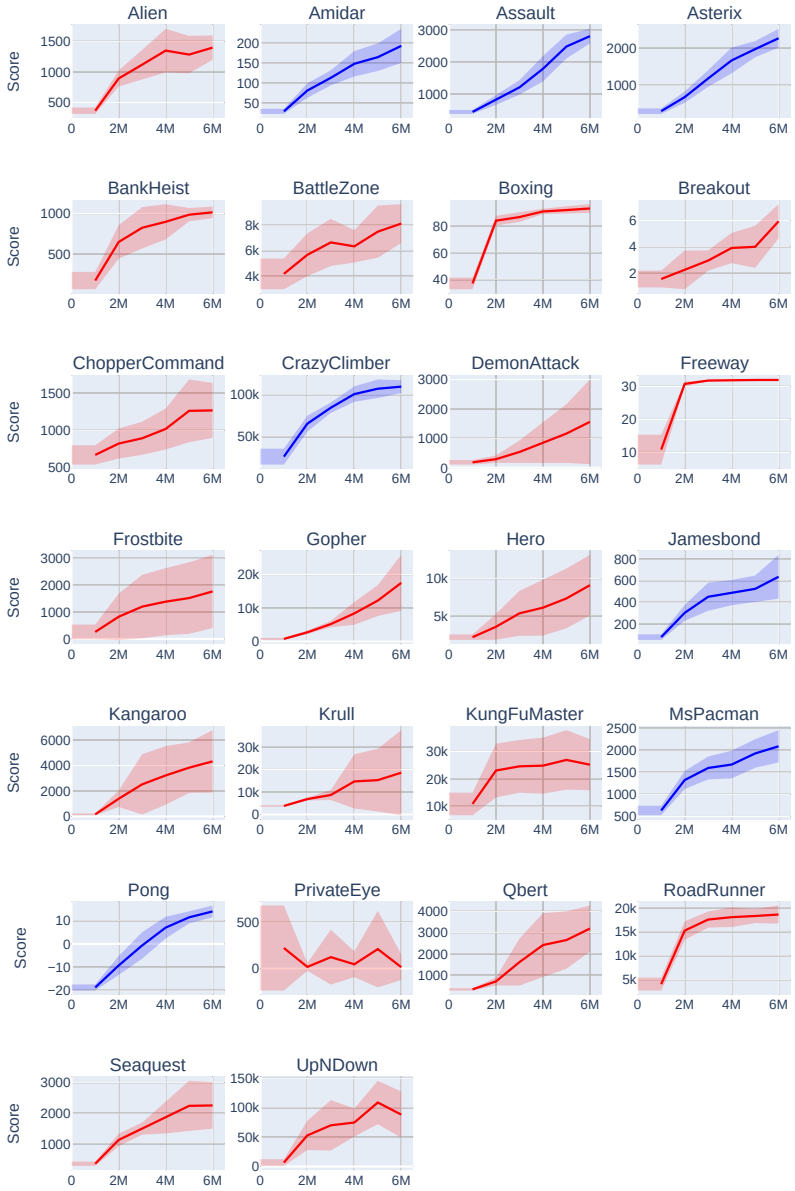

Figure 4.1: Performance curves for Atari-100k, with default settings. The values have been provided by the authors of the DreamerV2 paper. The x-axis represents the number of environment steps passed. The y-axis represents the validation score. Blue curves indicate environments, which we have selected for further evaluation.

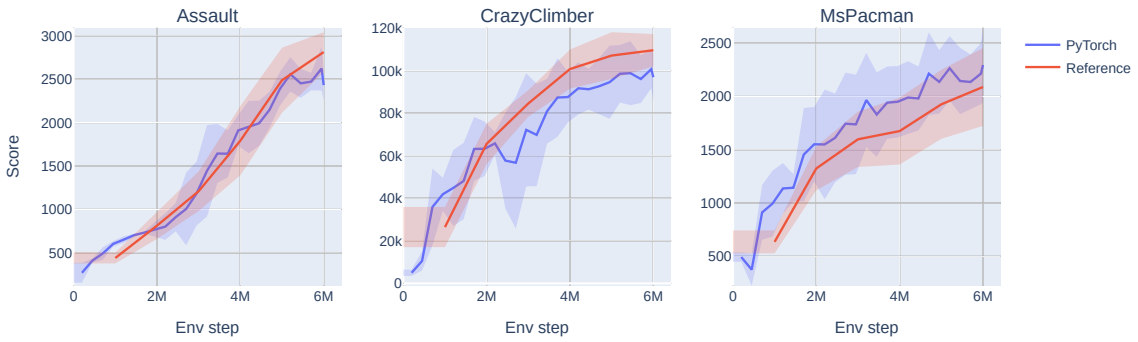

Figure 4.2: Comparison of the validation scores for Tensorflow and Pytorch versions. *Note:* The scores for the reference (Tensorflow) version have been supplied by the authors, and were not verified independently.

Figure 4.2 depicts the validation score curves for both the original implementation, and the Pytorch version.

4.2. Training recipe optimization

As mentioned, many model-free algorithms can be made more efficient through training recipe-level improvements. We wish to extend these findings to the domain of model-based RL. This is, however, non-trivial, and requires further investigation.

First, there are purely technical difficulties. Let us, for example, consider scaling up the frequency of model and RL updates. As opposed to the model-free case, we are dealing with two separate learning processes, that of the model and of the RL module, resulting in two separate update frequencies to be tuned. If we were to search for the optimal configuration through a grid search, we would be faced with a quadratic number of training runs to perform, relative to the model-free case, which would quickly become infeasible. It is therefore imperative to reduce the extent of the hyperparameter sweep as much as feasible.

Next, insofar as the collapse of the performance for regular model-free RL agents, when increasing the update-to-data frequency, is attributed to plasticity loss, it is not immediately clear whether model-based agents undergo the same process, whether the model and the RL module suffer from the primacy bias or perhaps only one of them does, and whether interventions proposed in the literature prove beneficial here as well. If such interventions require tuning, as is for example the case for periodic resets, then we possibly encounter another computationally expensive grid search.

4.3. Simple speedup test

We begin with an experiment involving a straightforward increase of the frequency of optimization steps, without any further changes. The goals are as follows: to establish a baseline for further analysis, to find the update-to-data regime where the agent performance degrades, and to investigate what happens at that point with the networks in question.

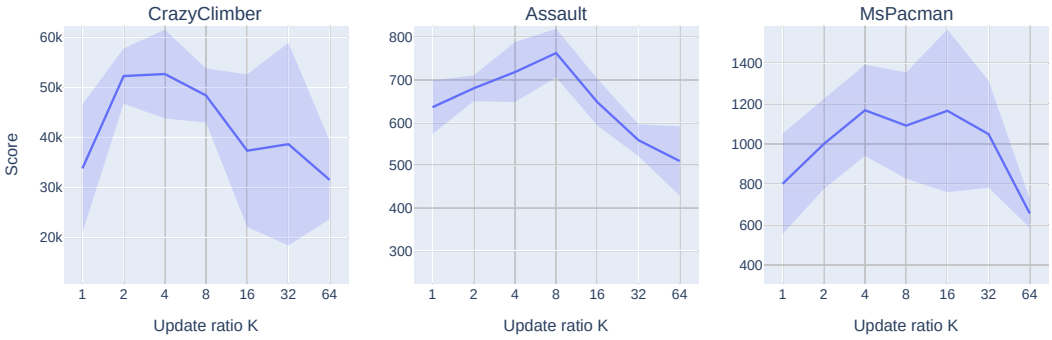

Figure 4.3: Final test episode returns for each of the 3 tested environments, relative to the data-to-update ratio E. The line indicates the mean across the RNG seeds, and the shaded area indicates the standard deviation. *Note*: The x-axis is logarithmic.

Setup Base DreamerV2 training recipe begins with a prefill of $P = 200 \times 10^3$ and continues until the total step count is $T = 200 \times 10^6$. The latter loop consists of $E = 64$ env steps (at the frame skip value of 4, this corresponds to 16 agent steps) and a single optimization step.

We replace T by 400×10^3 to match Atari-100k benchmark environment budget, and P with 20×10^3. Then, we perform tests with varying values of $E \in \{64, 32, 16, 8, 4, 2\}$, for a set of 3 selected environments and 5 RNG seeds.

Beyond that, for testing purposes, we also add an online train-validation split mechanism: each episode in the replay buffer is assigned either to the training set or the validation set – to be precise, every $(1/p)$-th episode is assigned to the validation set, where $p = 0.15$.

Results Let us first analyze final test performance, depending on the data-to-update ratio E, for each environment (Figure 4.3). The mean final score is maximized for a given region of the values of E, beyond which it decreases, presumably either due to under- or overoptimization. For each task, the optimal value is different: $2 - 4$ for `CrazyClimber`, ≈ 8 for `Assault` and a wide range for `MsPacman`.

4.3.1. Decoupling model and RL optimization

As noted previously, the optimal training schedules for the model and the RL module need not coincide - we may, for example, find that the model is overfitting, yet the RL head remains underoptimized. The next experiment aims to investigate the empirical effects of decoupling model and RL update frequencies.

Setup We adjust the code, so that the model is optimized once every K_{WM} environment steps, and the RL module is optimized once every K_{RL} environment steps. A grid search is performed for `Assault` environment [Extend to all three, if there's enough compute time], with ratios $K_{\text{WM}}, K_{\text{RL}} \in \{2, 4, 8\}$, corresponding to the observed regime, where the performance starts decreasing.

Results Figure 4.4 shows the agent performance depending on the configuration of the ratios. Both the model and the RL ratios play a significant role in determining the final score,

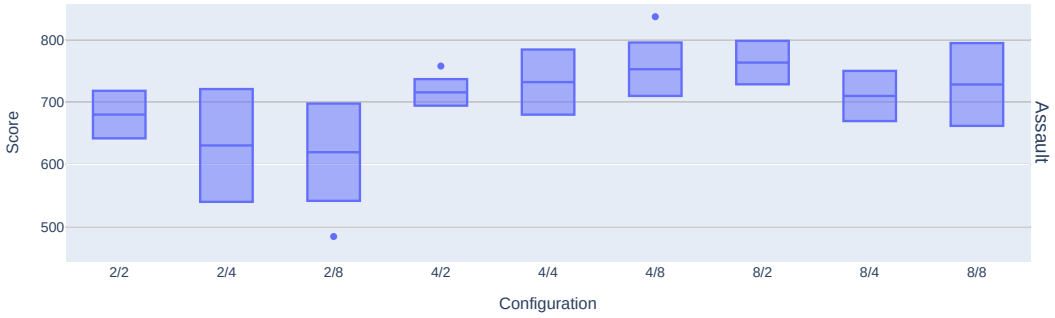

Figure 4.4: Final validation score for each model/RL update ratio configuration. The x-axis represents each run, marked by a tag in the form of '$K_{\mathrm{WM}}/K_{\mathrm{RL}}$'. The points represent the scores for each seed, with the corresponding box plot to the right.

though no clear pattern can be determined - for example, increasing RL update ratio does not necessarily improve the performance.

4.4. Investigating the learning dynamics of the agents

So far, the experiments suggest that achieving optimal or near-optimal performance may require, at the very least, a hyperparameter sweep over *both* model and RL update frequencies. Given the increase in the number of training runs this would require, we will now investigate methods to auto-select at least some of these parameters. In order to achieve this, we will start with an examination of the various metrics thoughout the run, and see whether they are correlated with the final score. The idea is that if we are able to control them throughout the optimization process, we may be able to guide them in such a way so as to achieve higher final performance.

4.4.1. Analyzing the world model

Since model learning is a supervised learning task, the most imporant metric to monitor is, we might presume, the validation loss. We compute it as follows: throughout the training process, we will load a batch of data using the validation episodes, and compute the mean model loss. We will start with the aggregate results - the relationship between the final validation loss and the final agent performance, for the equal-ratio experiment described in section 4.3. As we can see in Figure 4.5, it would appear, somewhat surprisingly, that the value, for individual trials, is not meaningfully correlated with the final score achieved by the agent. A closer inspection of the validation loss curves (Figure 4.6) reveals, that the agents do not, in general, exhibit classical overfitting behavior, in the sense that the validation loss curves plateau at worst, and in general the loss value keeps decreasing. The learning process nevertheless degrades, as the final values of the validation loss keep increasing as the ratio of the optimization and environment steps increases. The ratio values, for which the final loss value is the smallest, do not correspond to the ones which achieve the highest performance.

To further examine the behavior of the agent throughout the optimization process, let

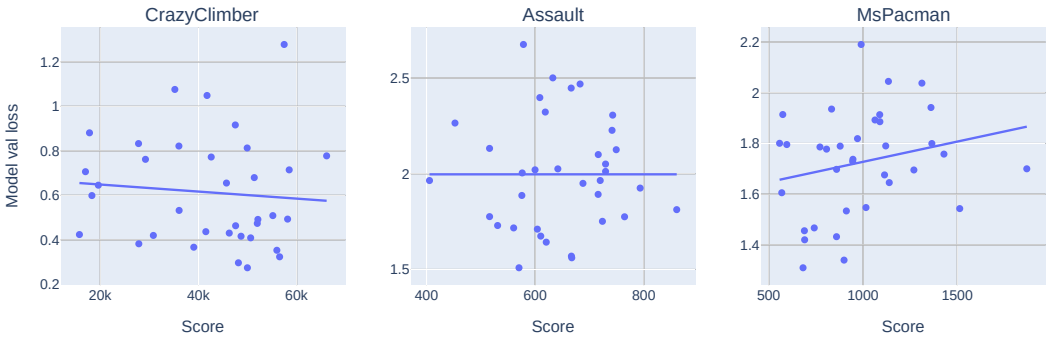

Figure 4.5: Relationship between final validation score and final model validation loss. Each point represents a single training run. The x-axis represents the agent performance. The y-axis represents the value of the model validation loss at the end of the training.

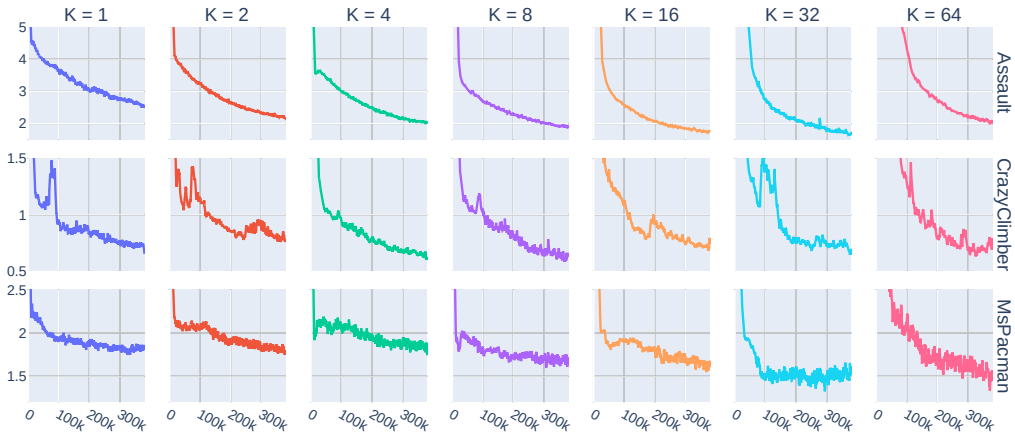

Figure 4.6: Averaged model validation loss curves for each configuration of the environment and the update ratio used. Rows represent different environments, and the columns the different update ratios used. The x-axis corresponds to the number of environment steps taken, and the y-axes represent loss values. The ranges of the y-axes are different for each environment.

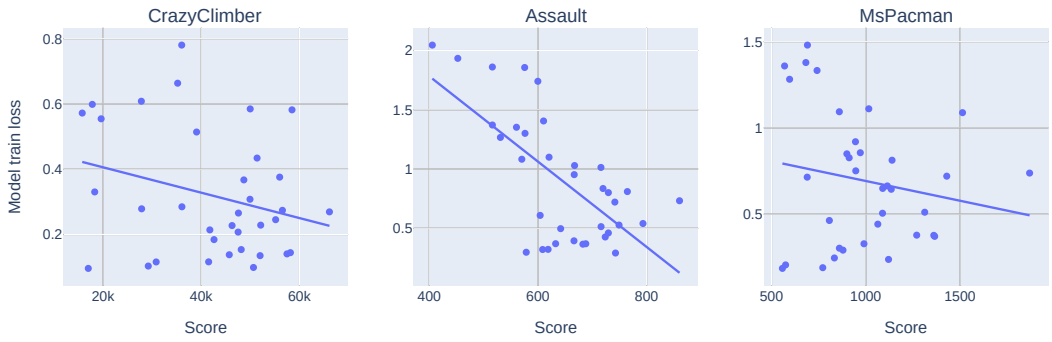

Figure 4.7: Relationship between final validation score and final model training loss. Each point represents a single training run. The x-axis represents the agent performance. The y-axis represents the value of the model training loss at the end of the training.

us take a look at the *training* loss curves. An aggregate comparison (Figure 4.7) reveals a degree of correlation between the agent performance scores and the training loss values. The loss curves (Figure 4.8) are of altogether different nature, compared to the validation loss curves. We can see a clear monotonic relationship between the final loss value achieved and the update frequency, though, as before, the better values do not lead to improvements in terms of the quality of the agents. Perhaps more interestingly, at very high update frequencies (i.e. small data-to-update ratios), the optimization process seems to stall out completely. For the value of $K = 1$, for example, in all three environments tested, near-"optimal" value is reached after the first 100×10^3 environment steps, whereafter it either improves marginally, or even increases.

4.4.2. Plasticity analysis

As we have noted previously, a number of works have identified the phenomenon of plasticity loss, meaning the inability of neural networks to learn from more recent data samples in the continual learning setting, as a possible culprit in the degradation of the RL agent performance. It might prove fruitful to investigate, whether something similar happens in the case of model-based RL, and specifically the agents based on Dreamer algorithm.

In the first order, we would like to perform a "post hoc" test of plasticity loss - the results will not allow us to monitor the occurence of the phenomenon *during* training, but will show whether the degradation has occured. We follow the strategy of [28], wherein an agent is trained *warm-started*, i.e. starting from the neural network parameters of a previously trained agent. In that study, the process is repeated 10 times. Due to computational constraints, we only perform a single repetition. Also, the environments under investigation there were drawn from the Coinrun suite of randomized environment. As we are interested in Atari tasks, and there is no standard way to randomize the environment - the idea of changing the Atari game does not work due to different action spaces, meaning different agent architectures required - we devise a custom randomization scheme for Atari. To be specific, at the start of the training run, we choose (1) a particular permutation σ of actions, meaning an action $a \in \{0, \ldots, A-1\} = \mathcal{A}$ is mapped to $\sigma(a) \in \mathcal{A}$; (2) whether to perform horizontal flip of the video frames; (3) whether to perform a vertical flip of the video frames. These randomizations

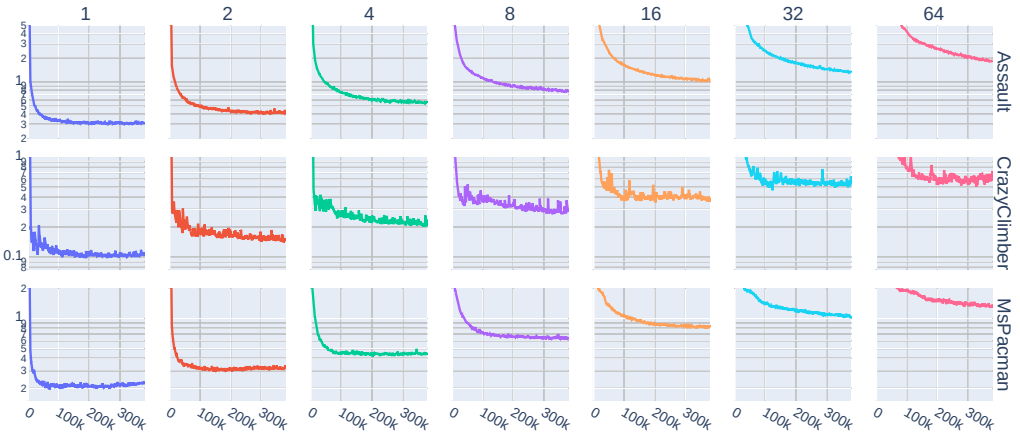

Figure 4.8: Averaged model training loss curves for each configuration of the environment and the update ratio used. Rows represent different environments, and the columns the different update ratios used. The x-axis corresponds to the number of environment steps taken, and the y-axes represent loss values. The ranges of the y-axes are different for each environment, and are **logarithmic** due to the wide range of values.

have been selected in order to ensure, that the warm-started agent approximately achieves the performance of a random agent, and that the observation and action statistics, e.g. the mean and standard deviation of the pixel values, remain the same.

The training recipe follows the reference test with the default DreamerV2 settings and an environment step budget of 6×10^6, used for comparing the Pytorch and Tensorflow implementations. The choice is motivated by the smaller variance of the results compared to the time horizon of 400×10^3 used for the other experiments. We load a checkpoint from a baseline test, with the update ratio value of $K = 1$, representing the checkpoints most affected by the primary bias. The environment used during the training run corresponding to the checkpoint is the same, but we change the RNG seeds to introduce further randomization.

The comparison of the warm-started and randomly-initialized agents (Figure 4.9) reveals, that at least in some instances the agents experience plasticity loss. To be specific, for `Assault` and `CrazyClimber`, we see either negligible or small impairment in terms of performance, whereas the effect is far more substantial for `MsPacman`. These findings are consistent with the analysis of the loss curves, since we have observed the stalling of the training process to be most pronounced for `MsPacman`. We also believe these results to support the assertion, that the devised Atari randomization scheme is effective for testing plasticity loss, in the sense that the fully trained agents start with a near-random performance, but the permutations and other environment changes do not impair the performance by themselves.

As opposed to the model-free case, in this instance we have two components, which may be responsible for the performance degradation. With this in mind, we will now perform a variation of the warm-start test, wherein only the RL component will have the network weights loaded from a previous checkpoint, and the world model will have the parameters initialized randomly. The other parameters of the experiment, such as the training loop structure, shall remain unchanged.

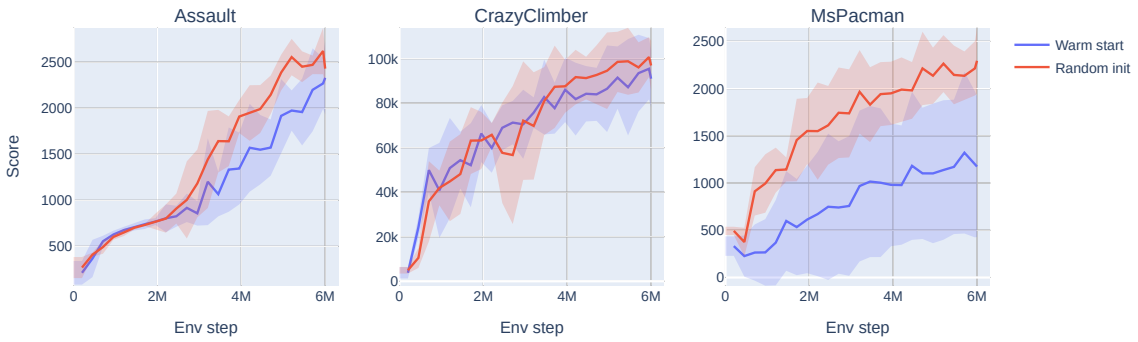

Figure 4.9: A comparison of the validation scores for the warm-started agents, and the ones with parameters initialized randomly. The lines show the mean and the standard deviation of agent validation scores, measured across 5 RNG seeds.

(Results for the actor-only warm start experiment)

4.4.3. Analyzing the RL component

Analysis of the behavior of the RL component is less straightforward, as there is no analogue of under- or overfitting of supervised models in the case of reinforcement learning. There are, however, a couple of phenomena, which can cause performance collapse, such as:

1. Value overestimation: Since most RL algorithms compute value estimates via bootstrapping, it is possible for bad initial value estimates to be amplified by the optimization process.

2. Model exploitation: In the context of model-based RL, and specifically the methods based on Dreamer architecture, since all the data given to the RL module is imagined by the model, inaccuracies thereof can cause the agent to learn wrong policies which perform well in the model-induced MDP.

3. Policy collapse: for actor-critic methods, without any regularization, the policy will tend to rapidly concentrate on the actions with the highest advantage estimates, which in turn would hurt exploration and prevent the agent from learning more optimal behaviors.

To investigate these, we will first take a look at the value curves. [?]

4.5. Auto-tuning update frequencies

It would therefore appear that these metrics cannot be directly used for the purposes of guiding the optimization process. There exist, however, works doing precisely that, which demands a further inquiry.

We will specifically do a closer examination of (Dynamic update ratios paper). The approach taken by the authors is to check validation loss throughout the training process, and to decrease the update frequency if the loss value has degraded, and increase it if it has improved. To start off, we will replicate the experiment, albeit with some notable changes:

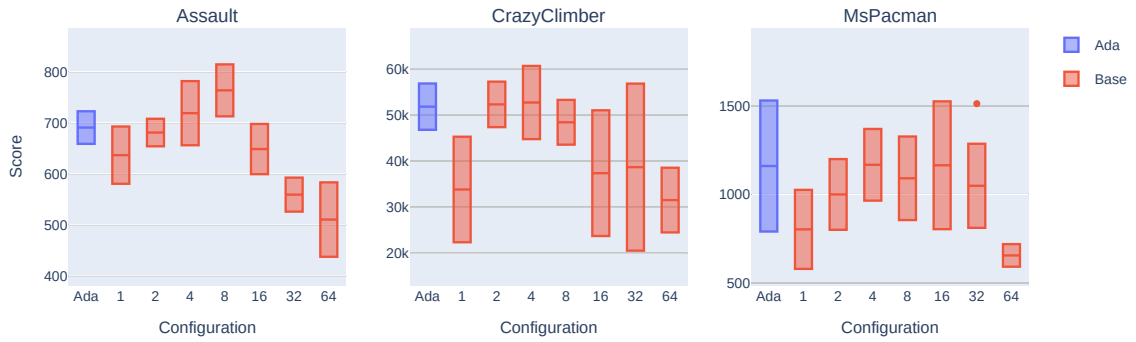

Figure 4.10: The comparison of the final validation scores, for the adaptive ratio experiment Ada and the baseline experiments 1 - 64, where the number denotes the model/RL update ratio K. Each subplot shows the results for a different environment tested. The bars signify mean and one standard deviation, obtained from a sample of 5 training runs for each configuration.

1. The authors propose using image reconstruction loss as the validation metric. We would rather have the method apply to models other than DreamerV2's specifically, however, so will use the total loss value.

2. A validation set is constructed differently: 3×10^3 transitions are collected every 100×10^3 steps, which corresponds to the flat 12% of the transitions being reserved for the validation set, whereas we reserve 15% of the *episodes* instead. Thus, the training set contains transitions which immediately precede or succeed the ones in the validation set - whether it represents a form of data contamination is unclear.

The results can be found in Figure 4.10, along with the baseline results for the sake of comparsion. It would appear that the strategy is, in general, effective, though not universal - for example, the scores on `Assault` do not quite reach the performance peak.

Given that the idea behind this approach is to minimize model overfitting by monitoring validation loss, our next step is to look at the validation loss. The results (Figure 4.11) do not seem to imply that the method has a consistent positive effect on the model validation loss. In fact, if we compare validation loss curves and the current ratio values (Figure 4.12), the relationship is not precisely what we would expect. As noted previously, the idea behind the method is to increase update ratio as long as the validation loss decreases, and vice versa. In short time scales, specifically the 2×10^3 environment steps between each ratio update, this does occur, but across longer time spans, the coupling breaks. For example, for `MsPacman` and the RNG seed of 1, the validation loss decreases up until the timestep of 200×10^3, yet the ratio keeps increasing. After the jump, presumably due to the influence of a new validation episode in the buffer, the loss value starts decreasing, and the ratio decreases.

4.5.1. Decoupled adaptive ratio system

It appears, then, that the theoretical underpinnings behind the effectiveness of this approach are not quite as clear as one would hope. Despite this, we believe one can nevertheless improve upon this method in a principled fashion. Drawing on the results concerning the decoupling

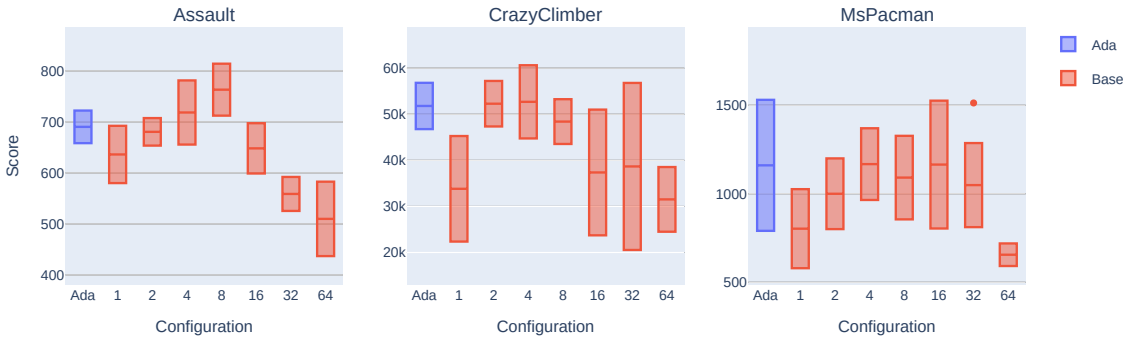

Figure 4.11: The comparison of the final model validation loss values, for the adaptive ratio experiment Ada and the baseline experiments 1 - 64, where the number denotes the model/RL update ratio K. Each subplot shows the results for a different environment tested. The bars signify mean and one standard deviation, obtained from a sample of 5 training runs for each configuration.

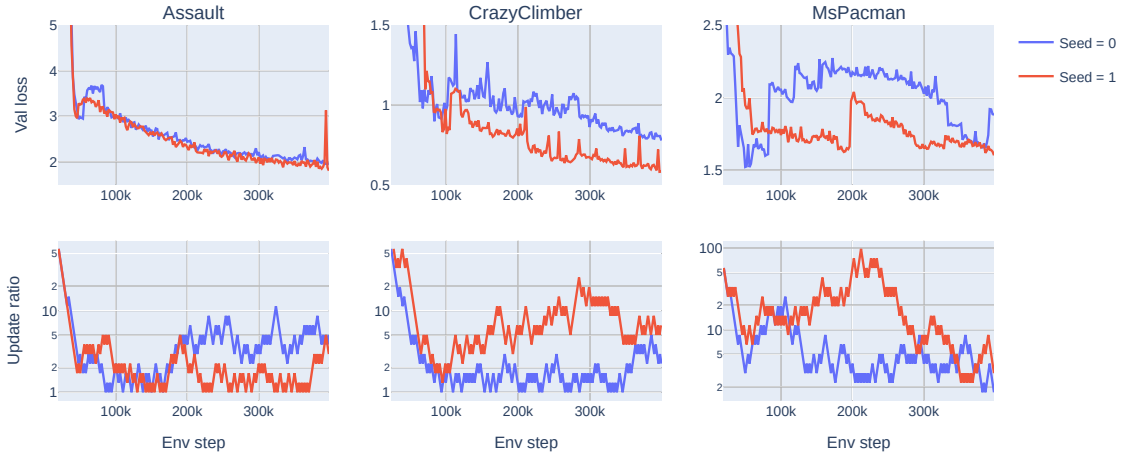

Figure 4.12: The curves for the model validation loss and the current update ratio, for the adaptive ratio experiment. Each column represents a different Atari task. Two training runs, corresponding to two different RNG seeds, are shown on each subplot. The model validation curves have been smoothed with exponential moving average to improve the clarity of the presentation.

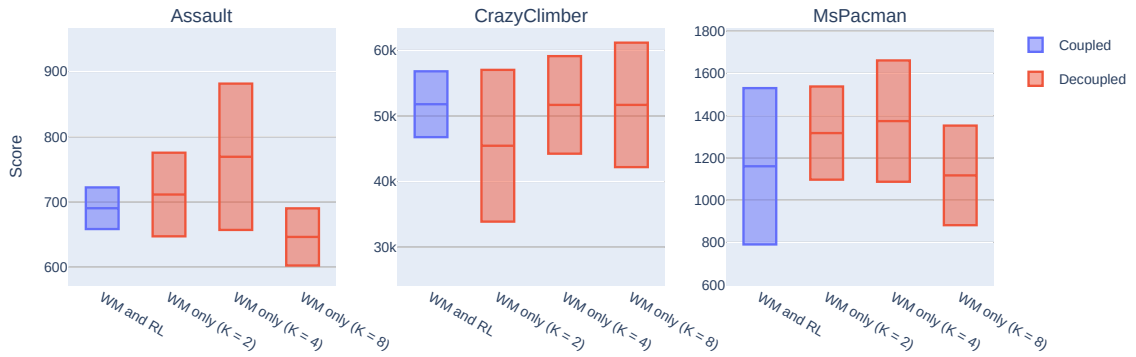

Figure 4.13: A comparison between the final validation scores for the coupled and the decoupled variants of the adaptive update ratio experiment. Each subplot represents a different environment tested. Each box signifies the results for a given configuration - mean and one standard deviation, obtained from a sample of 5 training runs for each configuration.

of the optimization rates for the model and the RL modules, we propose a similar decoupling in the adaptive regime. In particular, we will make only the model update ratio adaptive, and keep the RL update ratio fixed at a pre-defined value. The results (Figure 4.13) indicate, that the decoupling has a positive impact on the agent performance.

4.6. Data augmentation

Incorporating data augmentation into an RL framework can be done in non-trivial ways. DrQ, for example, combines transforming the input, averaging the Q functions across multiple augmented samples, and averaging the Q targets. In the context of model-based RL, one could develop similar schemes for both the world model and the RL modules, e.g. by averaging posterior state distributions over multiple image augmentations. This, however, would require changing the implementation of these modules, whereas our goal is to make the framework as modular as possible. Therefore, we opt to only augment the input, and forego the averaging of various quantities in question.

We follow the original paper in choosing random shifts as the image transformation to apply. The augmentations will be applied to both images in the batch for the model optimization, and the batch used to generate the initial states for the imagined rollouts. As far as the training recipe is concerned, we will start with a fixed-ratio setup in order to compare the data-augmented recipe with a non-data-augmented baseline.

Results The comparison of the test episode returns for the two variants (Figure 4.14) shows, that, with the base settings, the addition of data augmentation *degrades* performance, excepting perhaps `MsPacman`. As the main idea behind data augmentation is the reduction of model overfitting, we will next investigate the training and the validation loss curves for the model. The analysis of the final validation loss for each variant (Figure 4.15) reveals, that not only is it significantly higher compared to the baseline variant, but it also does not improve as the update frequency increases (equivalently, K decreases).

If we take a look at the model training loss curves (Figure 4.16), we may observe a following

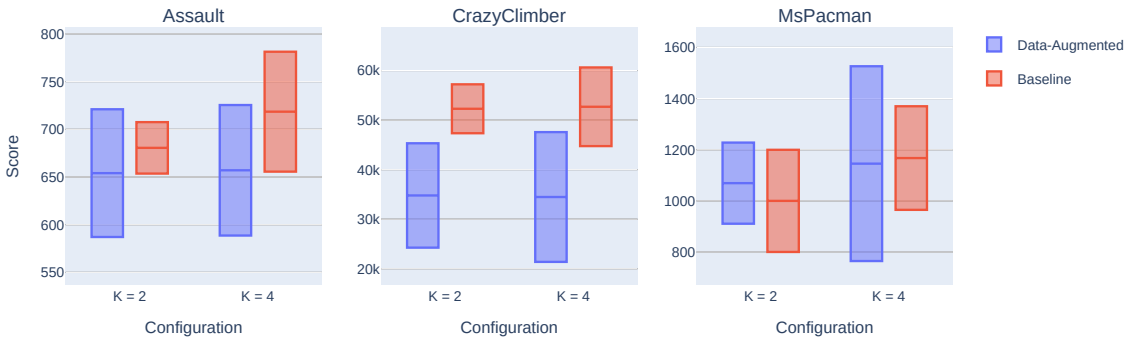

Figure 4.14: A comparison of the final validation scores for the data-augmented agent, and a non-data-augmented baseline. Each subplot represents the results for a single environment. In each subplot, results for different values of model/RL update ratio K are grouped together. Each group contains a box, signifying the mean and the standard deviation evaluated across 5 training runs, for the data-augmented and the non-data-augmented variants.

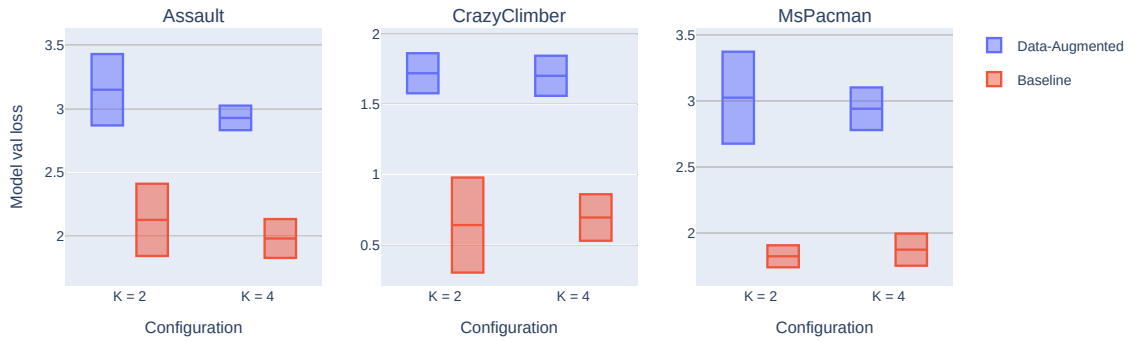

Figure 4.15: A comparison of the final model validation loss values for the data-augmented agent, and a non-data-augmented baseline. Each subplot represents the results for a single environment. In each subplot, results for different values of model/RL update ratio K are grouped together. Each group contains a box, signifying the mean and the standard deviation evaluated across 5 training runs, for the data-augmented and the non-data-augmented variants.

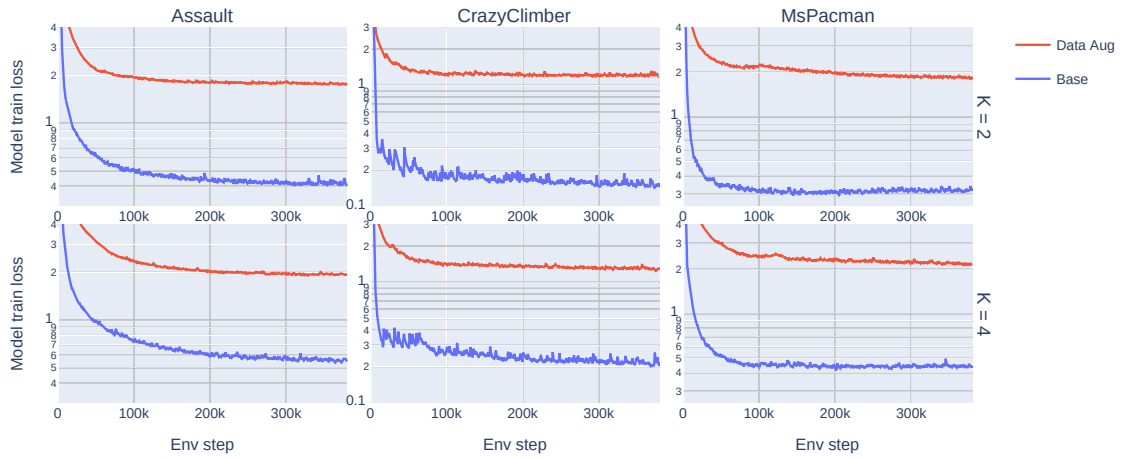

Figure 4.16: A comparison of the model training loss curves for the data-augmented agent, and a non-data-augmented baseline. Each column represents the results for a single environment. Each row corresponds to a different configuration - model/RL update ratio used. In each subplot, curves for the data-augmented and non-data-augmented variants are displayed. The curves have been smoothed by exponential moving average, to improve the clarity of the presentation.

phenomenon: for `Assault` and `CrazyClimber`, the addition of random shifts appears to slow down, or even stall out the training process, as shown by the slower rate of decrease (in log-space). For `MsPacman`, however, the reverse is true: in the original experiment, training loss stops improving at a certain point, whereas in the data-augmented scenario, we observe a consistent decrease. [What further conclusions to draw?]

As suggested in (DrQ paper citation), the type of image augmentation used has a large impact on the effectiveness of the method. To this end, the next experiment will assess the impact of changing the kind of the augmentation. We mostly follow the source paper in the selection of the transforms to test:

1. `none`: Baseline/No augmentations applied.
2. `shift4`: (Default) Apply a random image shift. The shift in each axis is selected randomly from $\{-4, \ldots, 4\}$.
3. `shift2`: Apply a random image shift. The shift in each axis is selected randomly from $\{-2, \ldots, 2\}$. This variant of `shift4` is motivated by the fact, that DrQ image size is 84×84, whereas we operate on images of size 64×64 - it could be possible, that the shift of 4 is too large.
4. `cutout`: Perform random erasing with probability $p = 0.5$.
5. `intensity`: Perform random brightness/intensity increase. To be specific, each pixel's value is multiplied by $f \sim \mathcal{U}[0.95, 1.05]$.
6. `rotate`: Rotate the image by $d \sim \mathcal{U}[-5, 5]$ degrees.

Figure 4.17: The final validation scores for different data augmentation types. Each row contains the results for a different environment tested. Each of the boxes corresponds to a given augmentation type, and displays the mean and the standard deviation measured with a sample of 5 training runs.

7. `vflip`: Flip the image vertically, with probability $p = 0.1$.

where $x \sim \mathcal{U}[a, b]$ denotes a sample from the uniform distribution over $[a, b] \subset \mathbb{R}$.

The results (Figure 4.17) suggest, that no type of image augmentation has a consistent positive effect on the agent performance.

4.7. Choice of the RL algorithm choice

So far, all the experiments have used the default RL module of DreamerV2, namely a variant of Advantage Actor-Critic (A2C). In this section, we shall investigate the effects of replacing it, whilst keeping the model component intact. Beyond hopefully achieving improvement in terms of agent performance, our objective is to elucidate the design choices when incorporating model-free RL algorithms into a model-based setting.

4.7.1. PPO Experiments

We will start with the experiments involving PPO. For the choice of the hyperparameters, we take the approach of replicating the default ones for A2C, whenever applicable. This includes the network architectures of the encoders for the actor and the critic functions, the value of the discount factor γ and GAE discount λ, actor and critic optimizer learning rates, and the entropy penalty α.

There remain two hyperparameters to select: the number of update epochs N, meaning the amount of times each data batch item is used, and the number of minibatches for each update epoch M, being the number of splits of the data batch into minibatches used for each optimization step. Thus, the total number of parameter updates is NM. We may note that, for $N = M = 1$, the procedure is in fact equivalent to the original A2C algorithm.

Beyond that, we need to specify the training recipe. We leave the model update ratio fixed at $K_{\text{WM}} = 4$, and finetune RL update ratio K_{RL}, number of update epochs N and number of minibatches M. As the baseline, we select a scenario with $K_{\text{WM}} = K_{\text{RL}} = 4$. We will examine following configurations:

1. `8/4/1` ($K_{\text{RL}} = 8, N = 4, M = 1$) - We exchange less frequent RL optimization steps for a greater number of update epochs within each optimization step.
2. `8/8/1` ($K_{\text{RL}} = 8, N = 8, M = 1$) - A variant of the first configuration, with more update epochs.
3. `16/8/1` ($K_{\text{RL}} = 16, N = 8, M = 1$) - A more extreme version of the first preset, with an equal number of RL optimization steps, and greater data reuse.
4. `8/4/8` ($K_{\text{RL}} = 8, N = 4, M = 8$) - We split the batch into 8 minibatches, increasing the total number of parameter updates by $8\times$.

Results If we take a look at the final validation scores (Figure 4.18), we may infer that using PPO, in any of the tested configurations, has, on average, a positive effect on the performance. The relationship is not consistent, though - for example, on `MsPacman`, the configuration `16/8/1` underperforms `8/4/1`, whereas the opposite is the case on the other two environments.

In order for the comparison to be fair, we ought to make sure, that the total training time does not vastly exceed the baseline. As Figure 4.19 indicates, except for the preset with $M = 8$, the training time is either the same, or even lower, even though the number of RL optimization steps is higher. This is likely due to the smaller number of imaginary batches, that need to be constructed.

4.7.2. SAC Experiments

Let us now conduct the experiments with SAC algorithm. In the first order, let us discuss the extent of the hyperparameter sweep to conduct. A complete grid search over network architectures, optimizer parameters etc. is clearly infeasible. We take the approach of reusing A2C's actor architecture and optimizer, and using A2C's encoder and optimizer settings for the value function as a proxy for SAC's Q functions. As both algorithms use the target functions, we copy the target Q function update schedule from A2C - a full copy every 100 optimization steps, to be precise.

The most impactful hyperparameter left, then, is the choice of temperature/reward scale α. As the first step, we perform a random search over the value of the parameter. Ideally,

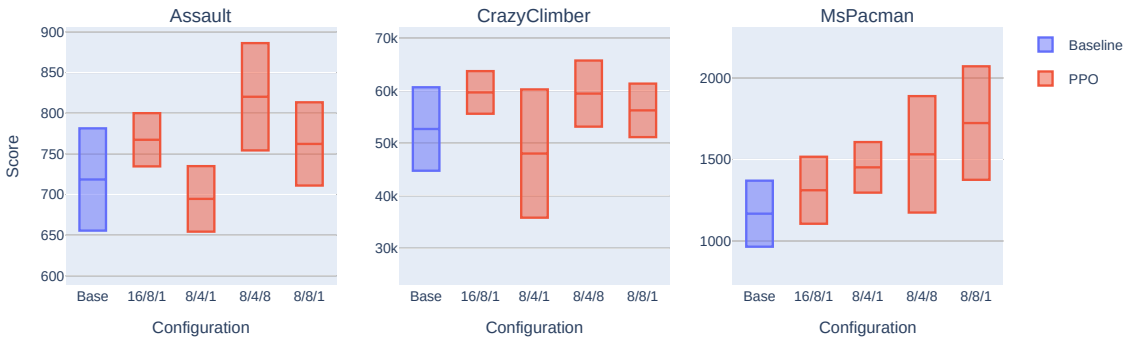

Figure 4.18: Comparison of the final validation scores between the baseline and a number of different configurations of PPO. Each subplot contains the results for a different Atari environment. Each box represents the mean and the standard deviation computed across 5 training runs.

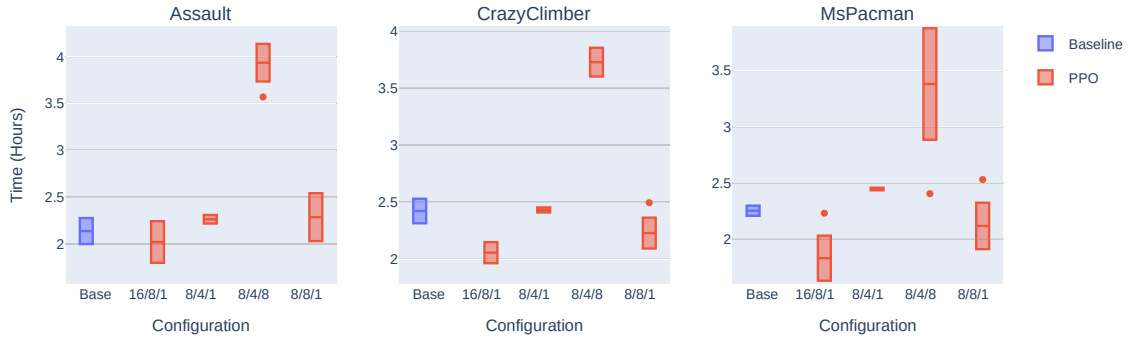

Figure 4.19: Comparison of the total training run durations for the baseline and a number of different configurations of PPO. Each subplot contains the results for a different Atari environment. Each box represents the mean and the standard deviation computed across 5 training runs.

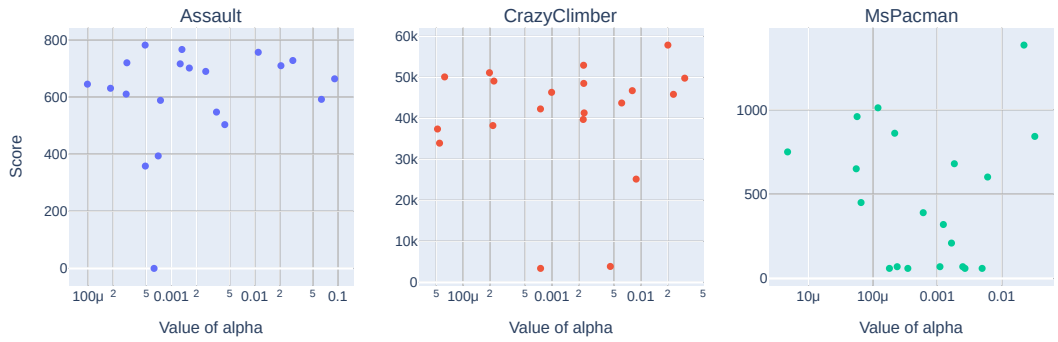

Figure 4.20: Relationship between SAC's α value, and the agent performance. Each subplot contains the results for a different Atari environment. Each point represents a single training run. The x-axis denotes the value of α parameter used in the test. The y-axis signifies the final validation score obtained by the agent.

we would perform a grid search, testing each value of α on the set of three environments and five RNG seeds. However, such an experiment would be computationally onerous. What we do instead, rather, is the following: for each pair of the environment/task and the RNG seed, we select the value of α from a log-normal distribution: $\log \alpha \sim \mathcal{N}(\log 10^{-3}, \log 10)$. In other words, we expect the first standard deviation to cover the interval $[10^{-4}, 10^{-2}]$. To increase the coverage, multiple α values can be tested for each pair. Then, we shall estimate the optimal value of α.

Results Let us start with the analysis of the final scores (Figure 4.20) Interestingly, the choice of α regularizer does *not* seem to significantly impact the performance. If we take a look at the final entropy values (Figure 4.21), which ought to be the primary metric controlled by the value of α, we may deduce that, while the value does impact the entropy (the effect can be seen most strongly on the chart concerning `Assault`), at a certain point we hit a lower bound of sorts, whereafter further decreasing the value of α does not affect it quite as strongly. The aforementioned lower entropy bound seems to be approximately 10^{-2} for all the environments.

Another method, which can be used to derive the value of α temperature, is auto-tuning, as described in (SAC Alg. and App. citation). Briefly, the idea is to specify a schedule of the mean policy entropy, rather than the α parameter itself, and optimize the value of α so that the schedule is achieved. The optimization is achieved via minimization of loss function $L = \alpha(H - H_0)$, where H_0 denotes the current entropy target.

In order to use it, we must specify the target entropy schedule. The recommendations for the model-free case suggest the use of a constant entropy target of $0.89H_{\max}$ for the discrete action spaces, where $H_{\max}$ denotes the maximum possible entropy for a policy in a given action space. However, the entropy curves of the best-performing agents in the baseline scenario (Figure 4.22) imply, that we ought to specify a rapidly decaying entropy schedule, with the final mean entropy value of approximately $0.05H_{\max}$.

With this in mind, we will now proceed with an experiment, wherein we will set the entropy target to follow the dotted line on Figure 4.22 - to be precise, we start at $0.99H_{\max}$,

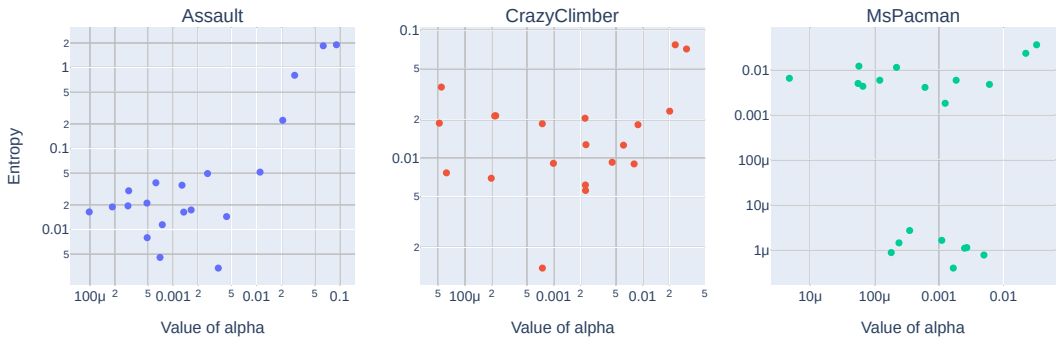

Figure 4.21: Relationship between SAC's α value, and the final mean policy entropy. Each subplot contains the results for a different Atari environment. Each point represents a single training run. The x-axis denotes the value of α parameter used in the test. The y-axis signifies the final value of the mean policy entropy metric.

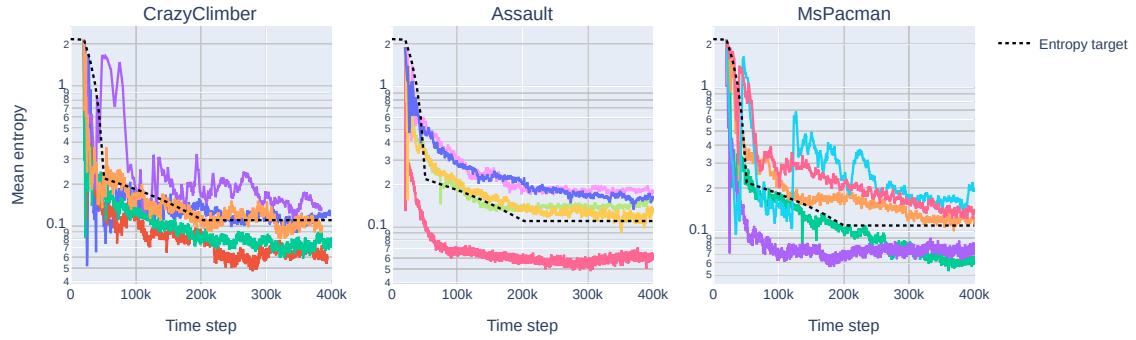

Figure 4.22: Mean policy entropy curves for the best-performing agents in the baseline experiment. Each Atari environment is represented by a different subplot. The x-axis denotes the number of environment step passed. The y-axis represents the mean policy entropy. Each solid line represents a different training run. The dashed line denotes an example entropy schedule approximating these schedules.

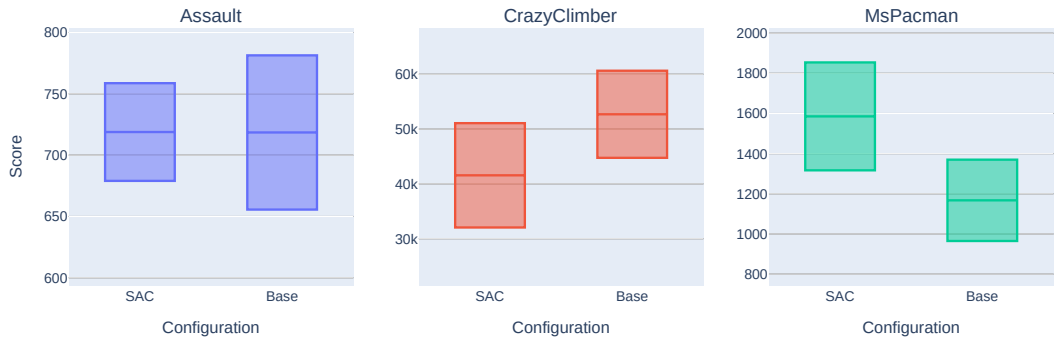

Figure 4.23: A comparison between the baseline experiment and the experiment with SAC and a user-defined entropy schedule. Each Atari environment tested is represented by a single subplot. The boxes denote the mean and the standard deviation of the final validation score, estimated with 5 training runs per configuration.

descend to $0.1H_{\max}$ by the environment step 50×10^3, and reach the target of $0.05H_{\max}$ by the environment step 200×10^3. As for the optimizer for the α loss value, we choose Adam with learning rate 2.5×10^{-3}, $\epsilon = 10^{-5}$ and leave other parameters as default in Pytorch.

The results (Figure 4.23) are inconclusive - performance on `MsPacman` is significantly improved, remains approximately the same on `Assault`, and is degraded on `CrazyClimber`.

4.8. Combining the improvements

Our experiments have singled out two major avenues for improvement, insofar as the agent performance is concerned: the adaptive update ratio schema, and the use of PPO instead of the default A2C. We have, however, tested them in isolation. It remains for us to see what happens if we employ them all at once. We shall also see the effect of these ablations on the 23 unseen environments.

Due to the requisite computational resources required to run the entire benchmark (26 games $\times$ 5 RNG seeds per single configuration), we will first examine the effects on the three test environments, select the hyperparameters to be used, and then run the benchmark on the chosen set.

4.8.1. Hyperparameter selection

In reference to the adaptive ratio system, the results suggest the use of a decoupled setup, wherein the RL update ratio is fixed. The performance gain is, however, dependent on the specific update ratio used. The validation scores obtained in the PPO experiment, on the other hand, indicate the use of either `16/8/1` or `8/8/1` configuration, the configuration name $K_{\text{RL}}/N/M$ denoting the RL optimization ratio, number of PPO epochs and number of PPO minibatches respectively. We exclude the experiments with $M > 1$ due to excessive adverse effect on the training speed, despite possibly yielding improved results.

These findings suggest a following setup: we will set the RL update ratio fixed, making the model update frequency alone adaptive, and focus on the hyperparameters for the PPO.

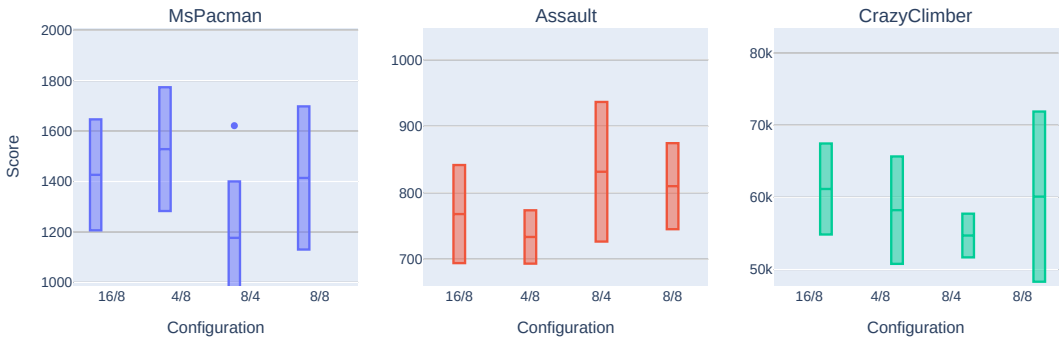

Figure 4.24: A comparison between the different hyperparameter presets for the experiment combining improvements in the context of a single agent.

We will test a slightly wider range of presets: `8/8/1`, `16/8/1`, `4/8/1` and `8/4/1`.

The scores obtained by the agents (Figure 4.24) appears inconclusive. None of the tested presets consistently outperforms the other ones - for example, `4/8` is the best on `MsPacman`, `8/4` on `Assault` and `16/8` on `CrazyClimber`. Another question that arises is whether the combination of the two improvements has yielded even better results, or, in other words, whether there has been any synergy between them. To this end, let us compare the results for the adaptive ratio, PPO and their combination side by side. As we can see (Figure 4.25), the integrated agent does *not* necessarily outperform the individual modifications. For example, on `MsPacman`, the PPO-only set of agents seem to achieve higher validation scores than the agents also utilizing the adaptive model ratios. It does, on the other hand, appear to slightly improve the overall results for the `Assault` and `CrazyClimber` Atari environments.

A more quantitative comparison (Table 4.1) supports the choice of either the preset $K_{\text{RL}} = 8, N = 8$, or $K_{\text{RL}} = 8, N = 4$, with both PPO and the adaptive model update ratio used, depending on whether we want to maximize mean or median human-normalized scores. In our estimation, the former configuration is more appropriate, due to lower gap between the mean and the median values.

4.8.2. Atari-100k benchmark

Now, we are in the position to perform the complete Atari-100k benchmark, consisting of 26 environments, as opposed to the subset of 3 video games most predictive of the overall performance investigated so far.

Setup We follow standard environment settings described in [29]. The main difference between the regular configuration is that for 200×10^6 environment steps, *sticky actions* are used, meaning an action is repeated with probability $p = 0.25$, whereas for Atari-100k benchmark, they are disabled. For each of the 26 Atari games, we perform 5 training runs, with different RNG seeds.

Results The complete results can be found in Table 4.2. We compare our agent's performance with a selection of sample-efficient architectures, both model-based (SimPLe, TWM, IRIS, DreamerV3), model-free (SR-SPR, BBF) and utilizing look-ahead search (EfficientZero).

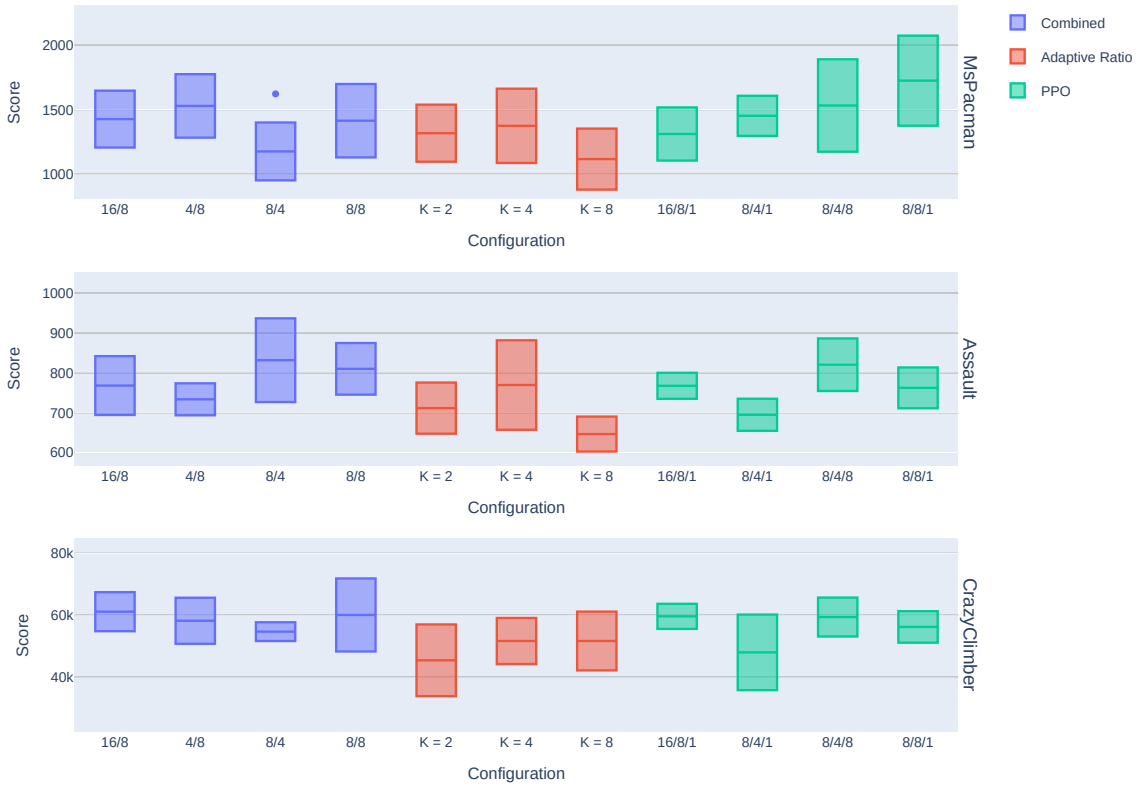

Figure 4.25: A comparison between the results for the adaptive ratio, PPO and their combination. Each row represents a single Atari environment. The boxes, denoting the mean and standard deviation computed across 5 RNG seeds, are placed into three groups, corresponding to the combined test, adaptive model ratio, and PPO. Each box in a given group corresponds to a different hyperparameter preset.

		Environment			Human Norm	
		Assault	CrazyClimber	MsPacman	Mean	Median
Ablation	Preset					
Adaptive	K = 2	711	45460	1317	0.83	0.94
	K = 4	769	51691	1374	0.95	1.05
	K = 8	646	51694	1117	0.86	0.82
Combined	16/8	767	**61163**	1426	1.08	1.05
	4/8	733	58230	1528	1.02	0.98
	8/4	**831**	54714	1176	1.02	**1.17**
	8/8	809	60104	1414	**1.09**	1.13
PPO	16/8/1	767	59651	1311	1.05	1.05
	8/4/1	694	48020	1451	0.86	0.91
	8/4/8	820	59431	1531	1.09	1.15
	8/8/1	762	56237	**1723**	1.02	1.04

Table 4.1: A numerical comparison of the results for different ablation types and hyperparameter presets. The middle three columns, under the header "Environment" contain the mean final validation scores. The rightmost two columns denote the human-normalized median and mean scores computed across the three environments. The bold font indicates highest values in each category.

The per-environment and aggregate human-normalized median and mean scores have been taken from [21] and [54].

Overall, the agent does not attain state-of-the-art performance, but, we argue, it performs better than might perhaps be expected. If we look at the human-normalized mean scores, we can see, that our method reaches performance comparable with the best model-based approach, DreamerV3. We also improve upon IRIS in the same metric. The human-normalized *median* performance, however, lags significantly behind DreamerV3 or BBF.

If we take a look at the validation score curves (Figure 4.26) (...)

Our solution is also more compute-efficient than the alternatives. A single training run takes approximately 3.5 hours running on half an A100 accelerator, compared with 0.1 A100-days for DreamerV3, 8 A100-hours for BBF, or 3.5 A100-days for IRIS.

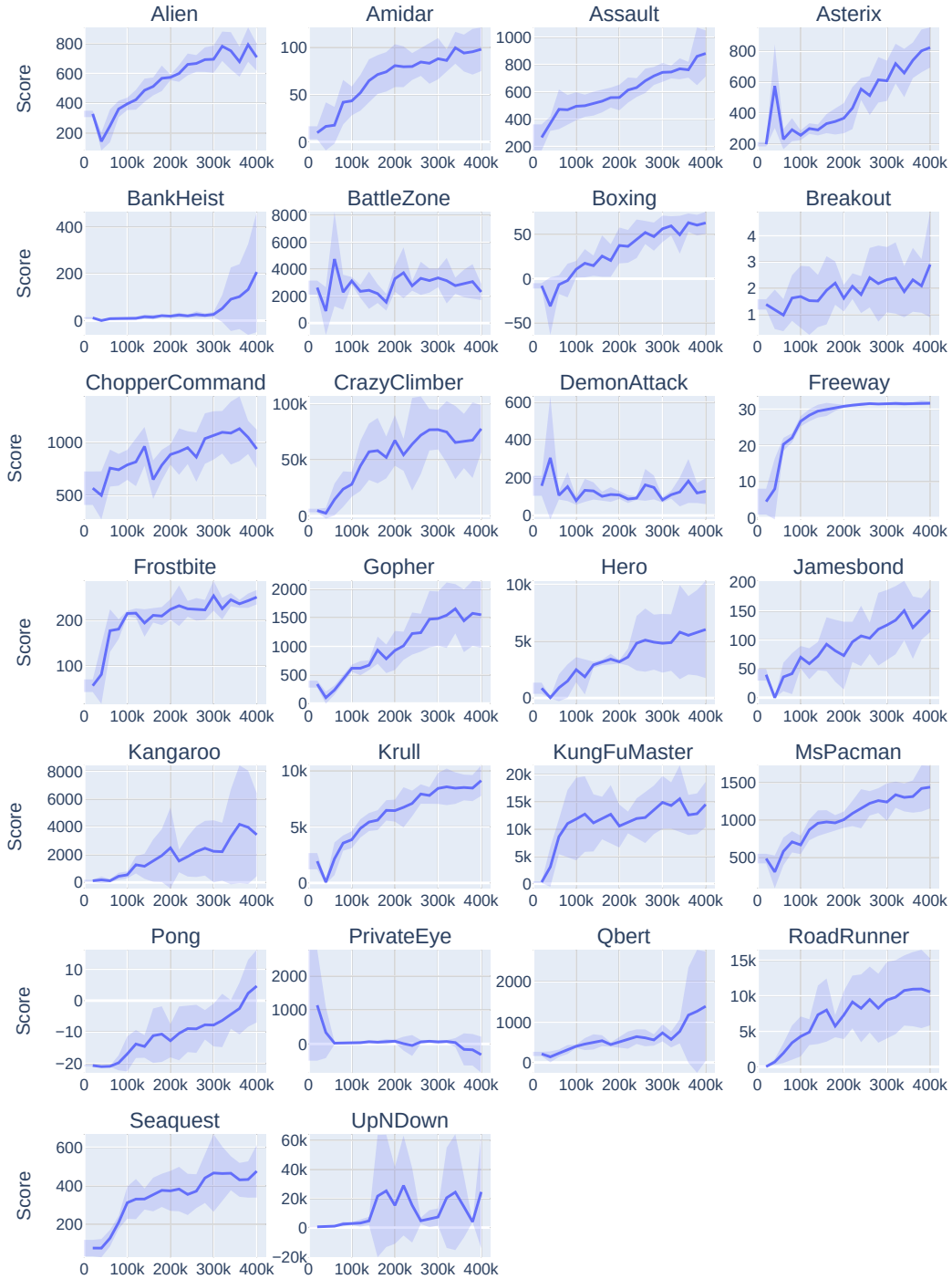

Figure 4.26: Atari-100k validation score curves.

Environment	Random	Human	SimPLe	TWM	IRIS	DreamerV3	SR-SPR	EfficientZero	BBF	Ours
Alien	228	7128	617	675	420	1118	1108	808	**1173**	708
Amidar	6	1720	74	122	143	97	203	149	**245**	98
Assault	222	742	527	683	1524	683	1089	1263	**2098**	881
Asterix	210	8503	1128	1117	854	1062	903	**25558**	3946	825
BankHeist	14	753	34	467	53	398	532	351	**733**	208
BattleZone	2360	37188	4031	5068	13074	20300	17671	13871	**24460**	2312
Boxing	0	12	8	78	70	82	46	53	**86**	63
Breakout	2	30	16	20	84	10	26	**414**	371	3
ChopperCommand	811	7388	979	1697	1565	2222	2362	1117	**7549**	937
CrazyClimber	10780	35829	62584	71820	59324	**86225**	45544	83940	58432	77748
DemonAttack	152	1971	208	350	2034	577	2814	13004	**13341**	128
Freeway	0	30	17	24	31	0	25	22	26	**32**
Frostbite	65	4335	237	1476	259	**3377**	2585	296	2385	251
Gopher	258	2412	597	1675	2236	2160	712	**3260**	1331	1551
Hero	1027	30826	2657	7254	7037	**13354**	8524	9316	7819	6029
Jamesbond	29	303	100	362	463	540	389	517	**1130**	152
Kangaroo	52	3035	51	1240	838	2643	3632	724	**6615**	3434
Krull	1598	2666	2205	6349	6616	8171	5912	5663	8223	**9130**
KungFuMaster	258	22736	14862	24555	21760	25900	18649	**30945**	18992	14549
MsPacman	307	6952	1480	1588	999	1521	1574	1281	**2008**	1443
Pong	-21	15	13	19	15	-4	3	**20**	17	5
PrivateEye	25	69571	35	87	100	**3238**	98	97	40	-317
Qbert	164	13455	1289	3331	746	2921	4044	**14448**	4447	1394
Seaquest	68	42055	683	774	661	962	819	1100	**1232**	476
UpNDown	533	11693	3350	15982	3546	46910	**112450**	17264	12102	24554
Human Median	0.000	1.000	0.130	0.510	0.289	0.490	0.685	**1.116**	0.917	0.205
Human Mean	0.000	1.000	0.330	0.960	1.046	1.250	1.272	1.945	**2.247**	1.210

Table 4.2: A comparison of Atari-100k scores for different models.

Chapter 5

Discussion

Bibliography

[1] Marc G. Bellemare, Will Dabney, and Rémi Munos. *A Distributional Perspective on Reinforcement Learning*. July 2017. arXiv: 1707.06887 [cs, stat]. (Visited on 07/23/2023).

[2] Marc G. Bellemare et al. *The Arcade Learning Environment: An Evaluation Platform for General Agents*. June 2013. DOI: 10.48550/arXiv.1207.4708. arXiv: 1207.4708. (Visited on 11/23/2024).

[3] James Bradbury et al. *JAX: composable transformations of Python+NumPy programs*. Version 0.3.13. 2018. URL: http://github.com/jax-ml/jax.

[4] Anthony Brohan et al. *RT-1: Robotics Transformer for Real-World Control at Scale*. Aug. 2023. DOI: 10.48550/arXiv.2212.06817. arXiv: 2212.06817 [cs]. (Visited on 11/24/2024).

[5] Pablo Samuel Castro et al. "Dopamine: A Research Framework for Deep Reinforcement Learning". In: (2018). URL: http://arxiv.org/abs/1812.06110.

[6] Xinlei Chen and Kaiming He. *Exploring Simple Siamese Representation Learning*. Nov. 2020. DOI: 10.48550/arXiv.2011.10566. arXiv: 2011.10566. (Visited on 11/24/2024).

[7] S. Chopra, R. Hadsell, and Y. LeCun. "Learning a Similarity Metric Discriminatively, with Application to Face Verification". In: *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*. Vol. 1. June 2005, 539–546 vol. 1. DOI: 10.1109/CVPR.2005.202. (Visited on 11/24/2024).

[8] Pierluca D'Oro et al. "Sample-Efficient Reinforcement Learning by Breaking the Replay Ratio Barrier". In: *The Eleventh International Conference on Learning Representations*. Sept. 2022. (Visited on 07/06/2024).

[9] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. May 2019. DOI: 10.48550/arXiv.1810.04805. arXiv: 1810.04805 [cs]. (Visited on 03/04/2023).

[10] Nicolai Dorka, Tim Welschehold, and Wolfram Burgard. *Dynamic Update-to-Data Ratio: Minimizing World Model Overfitting*. Mar. 2023. DOI: 10.48550/arXiv.2303.10144. arXiv: 2303.10144. (Visited on 10/28/2024).

[11] Gregory Farquhar et al. *TreeQN and ATreeC: Differentiable Tree-Structured Models for Deep Reinforcement Learning*. Mar. 2018. DOI: 10.48550/arXiv.1710.11417. arXiv: 1710.11417 [cs, stat]. (Visited on 04/02/2023).

[12] Meire Fortunato et al. *Noisy Networks for Exploration*. July 2019. DOI: 10.48550/arXiv.1706.10295. arXiv: 1706.10295 [cs, stat]. (Visited on 07/06/2024).

[13] Vincent François-Lavet, Raphael Fonteneau, and Damien Ernst. *How to Discount Deep Reinforcement Learning: Towards New Dynamic Strategies*. Jan. 2016. DOI: 10.48550/arXiv.1512.02011. arXiv: 1512.02011. (Visited on 11/24/2024).

[14] Spyros Gidaris, Praveer Singh, and Nikos Komodakis. *Unsupervised Representation Learning by Predicting Image Rotations*. Mar. 2018. DOI: 10.48550/arXiv.1803.07728. arXiv: 1803.07728. (Visited on 11/24/2024).

[15] Jean-Bastien Grill et al. *Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning*. Sept. 2020. DOI: 10.48550/arXiv.2006.07733. arXiv: 2006.07733. (Visited on 11/24/2024).

[16] David Ha and Jürgen Schmidhuber. "Recurrent World Models Facilitate Policy Evolution". In: *Advances in Neural Information Processing Systems*. Vol. 31. Curran Associates, Inc., 2018. (Visited on 11/24/2024).

[17] Tuomas Haarnoja et al. *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. Aug. 2018. DOI: 10.48550/arXiv.1801.01290. arXiv: 1801.01290 [cs, stat]. (Visited on 06/14/2023).

[18] Danijar Hafner et al. *Dream to Control: Learning Behaviors by Latent Imagination*. Mar. 2020. DOI: 10.48550/arXiv.1912.01603. arXiv: 1912.01603 [cs]. (Visited on 03/04/2023).

[19] Danijar Hafner et al. *Learning Latent Dynamics for Planning from Pixels*. June 2019. DOI: 10.48550/arXiv.1811.04551. arXiv: 1811.04551 [cs, stat]. (Visited on 03/04/2023).

[20] Danijar Hafner et al. *Mastering Atari with Discrete World Models*. Feb. 2022. DOI: 10.48550/arXiv.2010.02193. arXiv: 2010.02193 [cs, stat]. (Visited on 03/04/2023).

[21] Danijar Hafner et al. *Mastering Diverse Domains through World Models*. Apr. 2024. DOI: 10.48550/arXiv.2301.04104. arXiv: 2301.04104. (Visited on 11/24/2024).

[22] Hado van Hasselt, Arthur Guez, and David Silver. *Deep Reinforcement Learning with Double Q-learning*. Dec. 2015. DOI: 10.48550/arXiv.1509.06461. arXiv: 1509.06461. (Visited on 11/24/2024).

[23] Kaiming He et al. *Masked Autoencoders Are Scalable Vision Learners*. Dec. 2021. DOI: 10.48550/arXiv.2111.06377. arXiv: 2111.06377 [cs]. (Visited on 03/04/2023).

[24] Kaiming He et al. *Momentum Contrast for Unsupervised Visual Representation Learning*. Mar. 2020. DOI: 10.48550/arXiv.1911.05722. arXiv: 1911.05722. (Visited on 11/24/2024).

[25] Matteo Hessel et al. *Rainbow: Combining Improvements in Deep Reinforcement Learning*. Oct. 2017. DOI: 10.48550/arXiv.1710.02298. arXiv: 1710.02298 [cs]. (Visited on 03/04/2023).

[26] Shengyi Huang et al. "CleanRL: High-quality Single-file Implementations of Deep Reinforcement Learning Algorithms". In: *Journal of Machine Learning Research* 23.274 (2022), pp. 1–18. URL: http://jmlr.org/papers/v23/21-1342.html.

[27] Michael Janner et al. *When to Trust Your Model: Model-Based Policy Optimization*. Nov. 2021. DOI: 10.48550/arXiv.1906.08253. arXiv: 1906.08253 [cs, stat]. (Visited on 03/04/2023).

[28] Arthur Juliani and Jordan T. Ash. *A Study of Plasticity Loss in On-Policy Deep Reinforcement Learning*. May 2024. DOI: 10.48550/arXiv.2405.19153. arXiv: 2405.19153 [cs]. (Visited on 08/29/2024).

[29] Lukasz Kaiser et al. *Model-Based Reinforcement Learning for Atari*. Apr. 2024. DOI: 10.48550/arXiv.1903.00374. arXiv: 1903.00374 [cs, stat]. (Visited on 08/18/2024).

[30] Michael J. Kearns and Satinder P. Singh. "Bias-Variance Error Bounds for Temporal Difference Updates". In: *Proceedings of the Thirteenth Annual Conference on Computational Learning Theory*. COLT '00. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., June 2000, pp. 142–147. ISBN: 978-1-55860-703-3. (Visited on 11/24/2024).

[31] Diederik P. Kingma and Max Welling. *Auto-Encoding Variational Bayes*. Dec. 2022. DOI: 10.48550/arXiv.1312.6114. arXiv: 1312.6114. (Visited on 11/24/2024).

[32] Ilya Kostrikov, Denis Yarats, and Rob Fergus. *Image Augmentation Is All You Need: Regularizing Deep Reinforcement Learning from Pixels*. Mar. 2021. DOI: 10.48550/arXiv.2004.13649. arXiv: 2004.13649 [cs, eess, stat]. (Visited on 09/13/2024).

[33] Misha Laskin et al. "Reinforcement Learning with Augmented Data". In: *Advances in Neural Information Processing Systems*. Vol. 33. Curran Associates, Inc., 2020, pp. 19884–19895. (Visited on 11/24/2024).

[34] Timothy P. Lillicrap et al. *Continuous Control with Deep Reinforcement Learning*. July 2019. DOI: 10.48550/arXiv.1509.02971. arXiv: 1509.02971. (Visited on 11/24/2024).

[35] Ilya Loshchilov and Frank Hutter. "Decoupled Weight Decay Regularization". In: *International Conference on Learning Representations*. Sept. 2018. (Visited on 11/24/2024).

[36] Martín Abadi et al. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software available from tensorflow.org. 2015. URL: https://www.tensorflow.org/.

[37] Vincent Micheli, Eloi Alonso, and François Fleuret. *Transformers Are Sample-Efficient World Models*. Mar. 2023. DOI: 10.48550/arXiv.2209.00588. arXiv: 2209.00588 [cs]. (Visited on 03/04/2023).

[38] Volodymyr Mnih et al. "Human-Level Control through Deep Reinforcement Learning". In: *Nature* 518.7540 (Feb. 2015), pp. 529–533. ISSN: 1476-4687. DOI: 10.1038/nature14236. (Visited on 11/24/2024).

[39] Evgenii Nikishin et al. *The Primacy Bias in Deep Reinforcement Learning*. May 2022. DOI: 10.48550/arXiv.2205.07802. arXiv: 2205.07802 [cs, stat]. (Visited on 07/06/2024).

[40] Yazhe Niu et al. *DI-engine: A Universal AI System/Engine for Decision Intelligence*. https://github.com/opendilab/DI-engine. 2021.

[41] Mehdi Noroozi and Paolo Favaro. *Unsupervised Learning of Visual Representations by Solving Jigsaw Puzzles*. Aug. 2017. DOI: 10.48550/arXiv.1603.09246. arXiv: 1603.09246. (Visited on 11/24/2024).

[42] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. *Neural Discrete Representation Learning*. May 2018. DOI: 10.48550/arXiv.1711.00937. arXiv: 1711.00937 [cs]. (Visited on 03/04/2023).

[43] OpenAI et al. *Dota 2 with Large Scale Deep Reinforcement Learning*. Dec. 2019. DOI: 10.48550/arXiv.1912.06680. arXiv: 1912.06680 [cs, stat]. (Visited on 03/04/2023).

[44] OpenAI et al. *Solving Rubik's Cube with a Robot Hand*. Oct. 2019. DOI: 10.48550/arXiv.1910.07113. arXiv: 1910.07113. (Visited on 11/24/2024).

[45] Long Ouyang et al. "Training Language Models to Follow Instructions with Human Feedback". In: *Advances in Neural Information Processing Systems*. Oct. 2022. (Visited on 11/24/2024).

[46] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Dec. 2019. DOI: 10.48550/arXiv.1912.01703. arXiv: 1912.01703. (Visited on 11/23/2024).

[47] Luis Pineda et al. *MBRL-Lib: A Modular Library for Model-based Reinforcement Learning*. Apr. 2021. DOI: 10.48550/arXiv.2104.10159. arXiv: 2104.10159. (Visited on 11/23/2024).

[48] Tom Schaul et al. *Prioritized Experience Replay*. Feb. 2016. DOI: 10.48550/arXiv.1511.05952. arXiv: 1511.05952. (Visited on 11/24/2024).

[49] Dominik Schmidt and Thomas Schmied. *Fast and Data-Efficient Training of Rainbow: An Experimental Study on Atari*. Nov. 2021. DOI: 10.48550/arXiv.2111.10247. arXiv: 2111.10247 [cs]. (Visited on 03/04/2023).

[50] Julian Schrittwieser et al. "Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model". In: *Nature* 588.7839 (Dec. 2020), pp. 604–609. ISSN: 0028-0836, 1476-4687. DOI: 10.1038/s41586-020-03051-4. arXiv: 1911.08265 [cs, stat]. (Visited on 03/04/2023).

[51] Florian Schroff, Dmitry Kalenichenko, and James Philbin. *FaceNet: A Unified Embedding for Face Recognition and Clustering*. June 2015. DOI: 10.48550/arXiv.1503.03832. arXiv: 1503.03832. (Visited on 11/24/2024).

[52] John Schulman et al. *Proximal Policy Optimization Algorithms*. Aug. 2017. DOI: 10.48550/arXiv.1707.06347. arXiv: 1707.06347 [cs]. (Visited on 05/02/2023).

[53] John Schulman et al. *Trust Region Policy Optimization*. Apr. 2017. DOI: 10.48550/arXiv.1502.05477. arXiv: 1502.05477 [cs]. (Visited on 05/02/2023).

[54] Max Schwarzer et al. *Bigger, Better, Faster: Human-level Atari with Human-Level Efficiency*. Nov. 2023. DOI: 10.48550/arXiv.2305.19452. arXiv: 2305.19452 [cs]. (Visited on 07/01/2024).

[55] Max Schwarzer et al. *Data-Efficient Reinforcement Learning with Self-Predictive Representations*. May 2021. DOI: 10.48550/arXiv.2007.05929. arXiv: 2007.05929 [cs, stat]. (Visited on 03/04/2023).

[56] Younggyo Seo et al. *Masked World Models for Visual Control*. Nov. 2022. DOI: 10.48550/arXiv.2206.14244. arXiv: 2206.14244 [cs]. (Visited on 03/04/2023).

[57] David Silver et al. "Mastering the Game of Go with Deep Neural Networks and Tree Search". In: *Nature* 529.7587 (Jan. 2016), pp. 484–489. ISSN: 1476-4687. DOI: 10.1038/nature16961. (Visited on 11/23/2024).

[58] David Silver et al. *The Predictron: End-To-End Learning and Planning*. July 2017. DOI: 10.48550/arXiv.1612.08810. arXiv: 1612.08810. (Visited on 11/24/2024).

[59] Aravind Srinivas, Michael Laskin, and Pieter Abbeel. *CURL: Contrastive Unsupervised Representations for Reinforcement Learning*. Sept. 2020. DOI: 10.48550/arXiv.2004.04136. arXiv: 2004.04136 [cs, stat]. (Visited on 06/21/2023).

[60] Richard S Sutton. "Reinforcement learning: An introduction". In: *A Bradford Book* (2018).

[61] Richard S Sutton et al. "Policy Gradient Methods for Reinforcement Learning with Function Approximation". In: *Advances in Neural Information Processing Systems*. Ed. by S. Solla, T. Leen, and K. Müller. Vol. 12. MIT Press, 1999.

[62] Richard S. Sutton. "Learning to Predict by the Methods of Temporal Differences". In: *Machine Learning* 3.1 (Aug. 1988), pp. 9–44. ISSN: 1573-0565. DOI: 10.1007/BF00115009. (Visited on 11/24/2024).

[63] Yuval Tassa et al. *DeepMind Control Suite*. Jan. 2018. DOI: 10.48550/arXiv.1801.00690. arXiv: 1801.00690. (Visited on 11/24/2024).

[64] Josh Tobin et al. *Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World*. Mar. 2017. DOI: 10.48550/arXiv.1703.06907. arXiv: 1703.06907. (Visited on 11/24/2024).

[65] Hado van Hasselt, Matteo Hessel, and John Aslanides. *When to Use Parametric Models in Reinforcement Learning?* June 2019. DOI: 10.48550/arXiv.1906.05243. arXiv: 1906.05243 [cs, stat]. (Visited on 03/04/2023).

[66] Ashish Vaswani et al. *Attention Is All You Need*. Dec. 2017. DOI: 10.48550/arXiv.1706.03762. arXiv: 1706.03762 [cs]. (Visited on 03/04/2023).

[67] Pascal Vincent et al. "Extracting and Composing Robust Features with Denoising Autoencoders". In: Proceedings of the 25th International Conference on Machine Learning. Jan. 2008, pp. 1096–1103. DOI: 10.1145/1390156.1390294.

[68] Oriol Vinyals et al. "Grandmaster Level in StarCraft II Using Multi-Agent Reinforcement Learning". In: *Nature* 575.7782 (Nov. 2019), pp. 350–354. ISSN: 1476-4687. DOI: 10.1038/s41586-019-1724-z. (Visited on 11/24/2024).

[69] Ziyu Wang et al. *Dueling Network Architectures for Deep Reinforcement Learning*. Apr. 2016. DOI: 10.48550/arXiv.1511.06581. arXiv: 1511.06581. (Visited on 11/24/2024).

[70] Jiayi Weng et al. "EnvPool: A Highly Parallel Reinforcement Learning Environment Execution Engine". In: *Advances in Neural Information Processing Systems*. Ed. by S. Koyejo et al. Vol. 35. Curran Associates, Inc., 2022, pp. 22409–22421. URL: https://proceedings.neurips.cc/paper_files/paper/2022/file/8caaf08e49ddbad6694fae067442ee21-Paper-Datasets_and_Benchmarks.pdf.

[71] R.V. Yampolskiy. *Artificial Intelligence Safety and Security*. Chapman & Hall/CRC Artificial Intelligence and Robotics Series. CRC Press/Taylor & Francis Group, 2018. ISBN: 978-0-8153-6982-0.

[72] Denis Yarats et al. *Mastering Visual Continuous Control: Improved Data-Augmented Reinforcement Learning*. July 2021. DOI: 10.48550/arXiv.2107.09645. arXiv: 2107.09645 [cs]. (Visited on 07/06/2024).

[73] Weirui Ye et al. *Mastering Atari Games with Limited Data*. Dec. 2021. DOI: 10.48550/arXiv.2111.00210. arXiv: 2111.00210 [cs]. (Visited on 03/04/2023).

[74] Richard Zhang, Phillip Isola, and Alexei A. Efros. *Colorful Image Colorization*. Oct. 2016. DOI: 10.48550/arXiv.1603.08511. arXiv: 1603.08511. (Visited on 11/24/2024).